



**NATIONAL
UNIVERSITY**

**College of Computing Information
Technologies**

FLEX Course Material



Object Oriented Programming

CCOBJPRG

Education that works.

by Emmanuel A. Ramirez



MODULE GOALS

- ☑ Modular and Reusable Code Design: OOP promotes modular code design through principles like encapsulation, inheritance, and polymorphism. This facilitates code reuse as classes can be easily extended and reused in different parts or projects.
- ☑ Abstraction and Problem-Solving: OOP encourages abstract thinking by modeling real-world entities as objects with defined properties and behaviors. This abstraction aids developers in identifying essential features and tackling complex problems by breaking them down into manageable components represented by objects and classes.
- ☑ Flexible and Extensible Software Architecture: OOP facilitates flexible and extensible software architectures by promoting loose coupling and high cohesion between modules. Through inheritance and polymorphism, developers can create hierarchies of classes for code reuse and extension without modifying existing code.

College of Computing Information
Technologies

FLEX Course Material

**Introduction to
OOP****Definition**

a programming paradigm that organizes code into objects with properties and behaviors, promoting modularity, reusability, and abstraction for efficient software development.



Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects. These objects are instances of classes, which serve as blueprints for creating objects. OOP brings modularity and reusability to your codebase, making it easier to manage and maintain as it scales in complexity. Let's dive into some key concepts and principles of OOP:



Objects

are the basic building blocks of OOP. They encapsulate data and behavior. For example, a car object can have data attributes like color, make, and model, and behavior methods like `start()`, `stop()`, and `accelerate()`.



Classes

a blueprint for creating objects. It defines the properties (attributes) and behaviors (methods) that all objects of that class will have.



Encapsulation

refers to the bundling of data (attributes) and methods (behaviors) that operate on the data into a single unit, i.e., a class. This helps in data hiding and prevents direct access to the internal state of an object from outside the class.



Inheritance

a mechanism that allows a class (subclass or derived class) to inherit properties and behaviors from another class (superclass or base class). This promotes code reuse and helps in creating a hierarchy of classes.



Polymorphism

allows objects of different classes to be treated as objects of a common superclass. It enables flexibility in programming by allowing the same method to have different implementations in different classes.



Abstraction

the concept of hiding complex implementation details and showing only the essential features of an object.



Association, Aggregation, and Composition

Association represents a relationship where all objects have their lifecycle and there is no owner. Aggregation is a specialized form of association where objects have their lifecycle but there is a "whole-part" relationship, like a car having wheels. Composition is a stronger form of aggregation where the parts are destroyed when the owning object is destroyed, like a car and its engine.

MODULE GOALS

- ☑ Data Protection and Security: Encapsulation allows for the protection of data by hiding the internal state of objects and providing controlled access through methods, reducing the risk of unintended modification or corruption and enhancing data security within a program.
- ☑ Code Flexibility and Maintenance: Encapsulation promotes code flexibility by decoupling the implementation details from the external interface, enabling easier maintenance and updates without affecting other parts of the codebase, leading to more robust and maintainable software systems.

College of Computing Information
Technologies

FLEX Course Material



Encapsulation

**Definition**

the bundling of data and methods that operate on that data into a single unit, known as a class, with the aim of hiding the internal state of an object and restricting direct access to it from outside, thus promoting data integrity and abstraction in programming.



Encapsulation is a fundamental concept in Object-Oriented Programming (OOP) that involves bundling data and methods that operate on that data within a single unit, typically a class. This allows for the hiding of the internal state of an object and controlling access to it through methods.



Sample Problem

Bank Account Management, Let's consider a simple problem of bank account management, where we need to create a BankAccount class with encapsulated properties and methods.

```
// Main.java
public class Main {
    public static void main(String[] args) {
        // Create a new bank account
        BankAccount account = new
        BankAccount("123456789", 1000.0);
        account.deposit(500.0); // Deposit funds
        account.withdraw(200.0); // Withdraw funds

        System.out.println("Current balance: " +
        account.getBalance());
    }
}

// BankAccount class encapsulating account information
// and operations
class BankAccount {
    private String accountNumber;
    private double balance;
    // Constructor to initialize account number and balance
    public BankAccount(String accountNumber, double
    balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }
}
```

```
// Method to deposit funds into the account
public void deposit(double amount) {
    if (amount > 0) {
        balance += amount;
        System.out.println(amount + " deposited
        successfully.");
    } else {
        System.out.println("Invalid amount. Deposit failed.");
    }
}

// Method to withdraw funds from the account
public void withdraw(double amount) {
    if (amount > 0 && balance >= amount) {
        balance -= amount;
        System.out.println(amount + " withdrawn
        successfully.");
    } else {
        System.out.println("Insufficient balance or invalid
        amount. Withdrawal failed.");
    }
}

// Method to get the current balance of the account
public double getBalance() { return balance; }

// Method to get the account number
public String getAccountNumber() { return
    accountNumber; }
}
```



Output

```
500.0 deposited successfully.
200.0 withdrawn successfully.
Current balance: 1300.0
```



Score

Learning Activity / Exercise # ____ Encapsulation

Name _____

Program-Year
Level _____

Date _____

Problem Statement:

You are tasked with developing a software system for managing a library. One of the key components of this system is the management of books within the library. Your task is to create a Java class to represent a Book, ensuring that its properties are encapsulated and accessible only through controlled methods.

Your Book class should have the following features:

1. Encapsulation: The properties of a book, including its title, author, and number of pages, should be encapsulated within the class, preventing direct access from outside.
2. Constructor: Implement a constructor that initializes the title, author, and number of pages of a book when an instance of the class is created.
3. Getter Methods: Provide getter methods to retrieve the title, author, and number of pages of a book. These methods should allow controlled access to the encapsulated properties, ensuring data integrity and abstraction.

Your solution should demonstrate how encapsulation promotes data hiding and controlled access, making the Book class modular, maintainable, and secure within the library management system.

Encapsulation (1-4):

- _____ (1 point): Little to no use of encapsulation; properties are accessible directly from outside the class.
- _____ (2 points): Basic encapsulation implemented; some properties may still be accessed directly, but most are encapsulated.
- _____ (3 points): Encapsulation implemented effectively; properties are encapsulated and inaccessible from outside the class.
- _____ (4 points): Advanced encapsulation implemented with access modifiers (private) and proper use of getter methods; all properties are encapsulated and inaccessible from outside the class.

Constructor (1-4):

- _____ (1 point): No constructor implemented or constructor does not initialize required properties.
- _____ (2 points): Basic constructor implemented; initializes some properties but may be incomplete or inconsistent.
- _____ (3 points): Constructor implemented effectively; initializes all required properties consistently and accurately.
- _____ (4 points): Advanced constructor implemented with optional parameters or parameter validation; provides flexibility and ensures data integrity.

Getter Methods (1-4):

- _____ (1 point): No getter methods implemented or getter methods are incomplete or inconsistent.
- _____ (2 points): Basic getter methods implemented; retrieves some properties but may not cover all required properties.
- _____ (3 points): Getter methods implemented effectively; retrieves all required properties accurately and consistently.
- _____ (4 points): Advanced getter methods implemented with optional features (e.g., defensive copying, lazy loading); provides enhanced functionality and abstraction.

Demonstration of Encapsulation (1-4):

- _____ (1 point): Little to no demonstration of how encapsulation promotes data hiding and controlled access.
- _____ (2 points): Basic demonstration of encapsulation principles; explanation lacks depth or clarity.
- _____ (3 points): Clear demonstration of how encapsulation promotes data hiding and controlled access; explanation is concise and well-supported.
- _____ (4 points): Advanced demonstration with detailed explanations of how encapsulation enhances modularity, maintainability, and security within the library management system.



Expected Output

Title: _____

Author: _____

Pages: _____

MODULE GOALS

- Understanding Polymorphism: Exploring both interface and implementation inheritance allows learners to grasp the concept of polymorphism more comprehensively. They learn how interfaces enable polymorphic behavior by allowing objects of different classes to be treated interchangeably based on a common interface, while implementation inheritance provides a mechanism for code reuse and the "is-a" relationship, further enhancing polymorphic capabilities in object-oriented programming.
- Mastery of Design Principles: Delving into interface and implementation inheritance equips learners with a deeper understanding of fundamental design principles such as encapsulation, abstraction, and separation of concerns. They comprehend how interfaces promote loose coupling and modularity by abstracting behavior from implementation details, while implementation inheritance facilitates code reuse and promotes a hierarchical class structure, enabling more maintainable and extensible software systems.

College of Computing Information
Technologies

FLEX Course Material



Inheritance

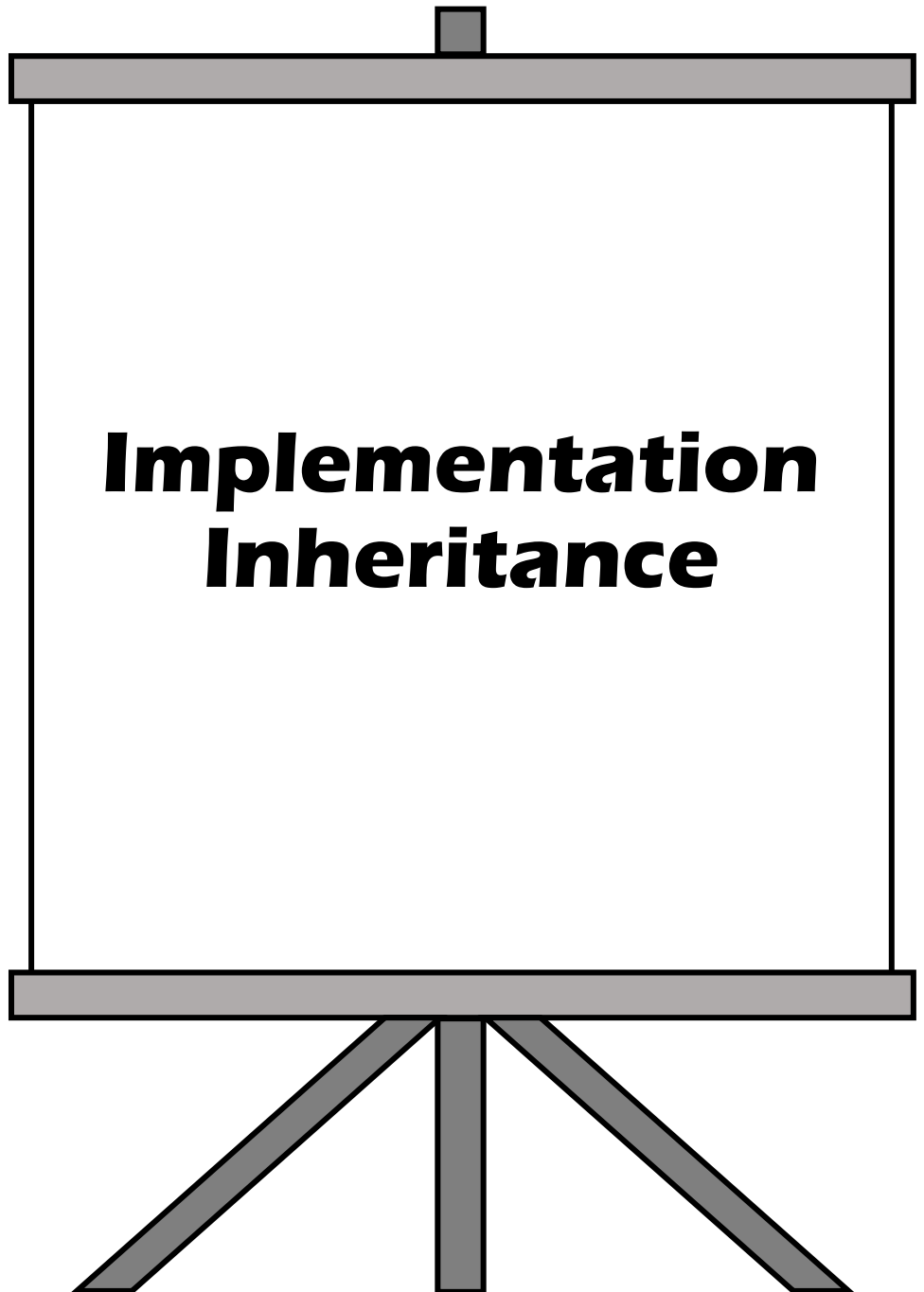
**Definition**

a fundamental concept in object-oriented programming where a new class (subclass or derived class) is created by inheriting the properties and behaviors (methods) of an existing class (superclass or base class). This allows the subclass to reuse the code already present in the superclass and also to add new functionalities or modify existing ones.

3.1

LEARNING OUTCOMES

- ☑ Understanding Code Reuse: Implementation inheritance enables developers to reuse existing code by inheriting both method signatures and implementations from a superclass. This promotes efficient development practices by avoiding redundant code and leveraging functionality already implemented in the superclass.
- ☑ Strengthening Class Relationships: Implementation inheritance reinforces the "is-a" relationship among classes, where a subclass is considered to be a more specialized version of its superclass. This understanding of class relationships is fundamental in designing and modeling object-oriented systems, contributing to better software architecture and design.



Definition

Implementation inheritance, a form of inheritance in object-oriented programming, involves a subclass inheriting both method signatures and implementations from its superclass. This facilitates code reuse, strengthens the "is-a" relationship among classes, and enhances modularity and scalability within the software system.



Implementation inheritance, occurring when a subclass inherits both method signatures and implementations from the superclass, facilitates code reuse and strengthens the "is-a" relationship among classes by inheriting behaviors and functionalities, thus enhancing modularity and scalability within the object-oriented programming paradigm.



Sample Problem

```
// Superclass
class Animal { public void makeSound() { System.out.println("Some generic sound"); } }

// Subclass demonstrating implementation inheritance
class Dog extends Animal {
    @Override
    public void makeSound() { System.out.println("Woof"); }
}

public class Main {
    public static void main(String[] args) {
        // Implementation inheritance
        Dog myDog = new Dog();
        myDog.makeSound(); // Outputs: Woof
    }
}
```



Output

Woof

3.2

LEARNING OUTCOMES

- ☑ Be able to write code that demonstrates different inheritance forms (single, multilevel, hierarchical) using appropriate syntax and concepts.
- ☑ Understand how to utilize inherited members (attributes and methods) within subclasses.
- ☑ Consider the trade-offs of inheritance complexity and the importance of maintaining code clarity.

Forms Inheritance



Definition

Inheritance in Java is a fundamental concept in object-oriented programming (OOP) that allows you to create new classes (subclasses) that inherit properties (attributes) and behaviors (methods) from existing classes (parent classes). This approach offers several benefits that contribute to well-structured, reusable, and maintainable code





Single Inheritance: This is the most basic form, where a subclass inherits from only one parent class. It's ideal for creating specialized objects based on a general foundation.



Sample Problem

```
// Parent class - Animal
public class Animal {
    protected String name; // Protected allows
    access from subclasses

    private int age; // Private restricts access to
    Animal class only

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void makeNoise() {
        System.out.println("Animal Noise!");
    }

    // Getter method for private attribute
    (optional)
    public int getAge() {
        return age;
    }
}
```

```
public class Dog extends Animal {
    private String breed;

    public Dog(String name, int age, String breed)
    {
        super(name, age); // Call parent constructor
        to initialize inherited attributes
        this.breed = breed;
    }

    // Override makeNoise() method for specific
    dog sound
    @Override
    public void makeNoise() {
        System.out.println("Woof!");
    }

    public String getBreed() {
        return breed;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy", 3, "Golden Retriever");
        System.out.println("Dog Name: " + myDog.name); // Inherited from Animal
        System.out.println("Dog Age: " + myDog.getAge()); // Access via getter method
        System.out.println("Dog Breed: " + myDog.getBreed());
        myDog.makeNoise(); // Calls overridden method in Dog class
    }
}
```



Output

```
Dog Name: Buddy
Dog Age: 3
Dog Breed: Golden Retriever
Woof!
```



Multilevel Inheritance: Here, a class inherits from another class, which itself inherits from yet another, forming a chain of inheritance. This allows for very specific object types derived from a broader base class. However, managing complex inheritance hierarchies requires careful planning to avoid confusion.



Sample Problem

```
// Grandparent class - Shape
public class Shape {
    public void draw() {
        System.out.println("Drawing a shape...");
    }
}

// Parent class - Polygon inherits from Shape
public class Polygon extends Shape {
    // No additional methods for this example
}

// Subclass - Triangle inherits from Polygon
public class Triangle extends Polygon {
    @Override
    public void draw() {
        System.out.println("Drawing a triangle..."); // Override draw() for specific behavior
    }
}

public class Main {
    public static void main(String[] args) {
        Triangle myTriangle = new Triangle();
        myTriangle.draw(); // Calls overridden method in Triangle class
    }
}
```



Output

Drawing a triangle...



Hierarchical Inheritance: Multiple subclasses inherit from a single parent class. This approach is useful for modeling related objects that share common functionalities but have their own variations. It promotes code reuse for the core functionalities while enabling specialization for each subclass.



Sample Problem

```
// Parent class - Animal
public class Animal {
    public void makeNoise() {
        System.out.println("Generic animal noise...");
    }
}

// Subclass - Dog inherits from Animal
public class Dog extends Animal {
    @Override
    public void makeNoise() {
        System.out.println("Woof!"); // Override for
dog sound
    }
}

// Subclass - Cat inherits from Animal
public class Cat extends Animal {
    @Override
    public void makeNoise() {
        System.out.println("Meow!"); // Override for
cat sound
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Animal(); //
Generic animal object
        myAnimal.makeNoise(); // Prints generic
message

        Dog myDog = new Dog();
        myDog.makeNoise(); // Calls overridden
method in Dog class (Woof!)

        Cat myCat = new Cat();
        myCat.makeNoise(); // Calls overridden
method in Cat class (Meow!)
    }
}
```



Output

```
Generic animal noise...
Woof!
Meow!
```



Multiple inheritance, in Java specifically, is a feature that allows a subclass to inherit properties (attributes) and behaviors (methods) from more than one parent class. While it offers potential benefits for code reuse, it can also introduce complexity and ambiguity.



Sample Problem

```
public interface Drawable {
    public void draw();
}

public interface Movable {
    public void move();
}

public class RedSquare implements Drawable,
Movable {

    @Override
    public void draw() {
        System.out.println("Drawing a red square...");
    }

    @Override
    public void move() {
        System.out.println("Moving the red
square...");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        RedSquare redSquare = new RedSquare();
        redSquare.draw(); // Output: Drawing a red
square...
        redSquare.move(); // Output: Moving the red
square...
    }
}
```



Output

```
Drawing a red square...
Moving the red square...
```



Hybrid inheritance in Java refers to combining different forms of inheritance (single, multilevel, hierarchical) within a single class structure. It allows you to create more complex class hierarchies that leverage the benefits of each inheritance type.



Sample Problem

```
public interface Drawable {  
    public void draw();  
}  
  
public class Animal {  
    protected String name; // Protected for access  
    from subclasses  
  
    public Animal(String name) {  
        this.name = name;  
    }  
  
    public void identify() {  
        System.out.println("I am an animal. My name  
is " + name);  
    }  
}
```

```
public class Fish extends Animal implements  
Drawable {  
  
    public Fish(String name) {  
        super(name); // Call parent constructor to  
initialize name  
    }  
  
    @Override  
    public void draw() {  
        System.out.println("Drawing a fish...");  
    }  
  
    @Override  
    public void identify() { // Method overriding  
(optional)  
        System.out.println("I am a fish. My name is "  
+ name);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Fish myFish = new Fish("Finny");  
        myFish.draw(); // Output: Drawing a fish...  
        myFish.identify(); // Output (overridden): I am a fish. My name is Finny  
    }  
}
```



Output

```
Drawing a fish...  
I am a fish. My name is Finny
```



NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES



Score

Learning Activity / Exercise # ____ Inheritance

Name

Program-Year
Level

Date

Problem Statement:

You are tasked with developing a program to manage various geometric shapes, including rectangles and circles. Each shape possesses unique characteristics and behaviors, such as specific dimensions and methods to calculate their area and perimeter. Your objective is to design a system that accurately computes the area and perimeter of each shape based on its properties, facilitating efficient geometric calculations in software applications.

Requirements:

1. **Shape Class:** Create a parent class named Shape that serves as the foundation for all geometric shapes. This class should include methods to calculate the area and perimeter of shapes, which will be overridden by subclasses.
2. **Rectangle Class:** Develop a subclass named Rectangle, which represents a rectangular shape. This class should contain attributes for length and width, along with methods to compute its area and perimeter based on these dimensions.
3. **Circle Class:** Implement another subclass named Circle, representing a circular shape. This class should include a radius attribute and methods to calculate its area and perimeter using the radius value.
4. **Main Program:** Create a main class named Main to test the functionality of the shape hierarchy. Instantiate objects of both Rectangle and Circle classes, providing specific dimensions (length, width, or radius) for each shape. Verify that the program accurately calculates and displays the area and perimeter for both shapes.

Ensure that the program adheres to object-oriented principles, such as encapsulation, inheritance, and method overriding, to promote code reusability and maintainability. Additionally, validate input parameters to handle potential errors gracefully, ensuring robustness and reliability in geometric calculations.



Expected Output

Rectangle Area: ____

Rectangle Perimeter: ____

Circle Area: ____

Circle Perimeter: ____

MODULE GOALS

- Flexibility in Code Design:
Polymorphism allows for the creation of code that can work with objects of various types, providing flexibility in code design. By treating objects of different classes through a common interface, developers can write code that is more adaptable to changes and can easily accommodate new types of objects without modifying existing code.
- Enhanced Code Readability and Maintainability:
Polymorphism promotes code readability by enabling more concise and intuitive code structures. By utilizing polymorphic behavior, developers can write code that focuses on the common behavior shared by different types of objects, rather than dealing with the specific implementation details of each individual type. This abstraction leads to clearer and more understandable code, making it easier to maintain and debug over time.

College of Computing Information
Technologies

FLEX Course Material



Polymorphism

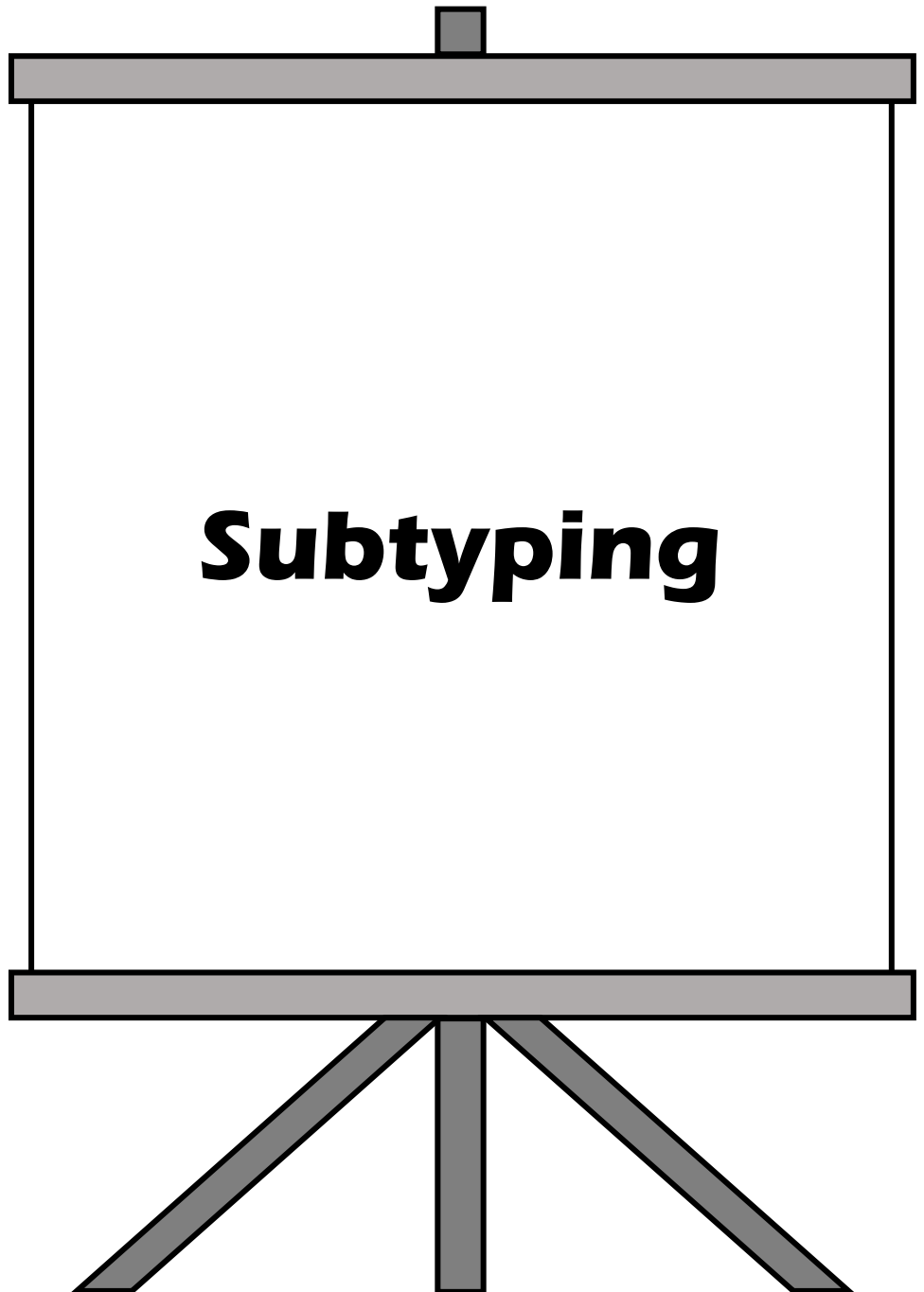
**Definition**

the ability of different objects to respond in unique ways to the same message or method call, allowing for flexibility and dynamic behavior in object-oriented programming.

4.1

LEARNING OUTCOMES

- ❑ Understanding of Polymorphism: Subtyping facilitates polymorphism in object-oriented programming, allowing objects of different subclasses to be treated interchangeably through a common superclass or interface. Learners gain insight into how subtyping enables code reuse and flexibility by promoting a uniform interface for interacting with objects of varying types.
- ❑ Mastery of Inheritance Hierarchies: Through studying subtyping, learners develop a deep understanding of inheritance hierarchies, including superclass-subclass relationships and the concept of "is-a" relationships. They grasp the importance of designing class hierarchies that accurately model real-world entities and behaviors, leading to more efficient and maintainable codebases.



Definition

Subtyping refers to the relationship between types where a subtype is a valid substitute for its supertype, allowing for polymorphic behavior and code reuse in object-oriented programming.



Preconditions: Preconditions are requirements that must be met before a method is executed. They define the state of the system or object that must be true for the method to function properly. By specifying preconditions, developers ensure that methods are invoked under suitable conditions, preventing unexpected behavior.



Sample Problem

Define a method called `divide` that takes two integers as parameters and returns the result of dividing the first integer by the second. However, ensure that the second integer is not zero before performing the division.

```
public class Divider {  
    public static int divide(int dividend, int divisor) {  
        // Check precondition: divisor must not be zero  
        if (divisor == 0) {  
            throw new IllegalArgumentException("Divisor cannot  
be zero.");  
        }  
  
        // Perform division  
        return dividend / divisor;  
    }  
  
    public static void main(String[] args) {  
        int dividend = 10;  
        int divisor = 2;  
  
        // Invoke divide method with suitable precondition  
        int result = divide(dividend, divisor);  
        System.out.println("Result of division: " + result);  
    }  
}
```

In the `divide` method, we first check the precondition by ensuring that the divisor is not zero. If the precondition is violated and the divisor is zero, we throw an `IllegalArgumentException` to indicate that the method cannot be executed under such conditions. By checking and enforcing preconditions, we ensure that the `divide` method is invoked under suitable conditions, preventing unexpected behavior such as division by zero.



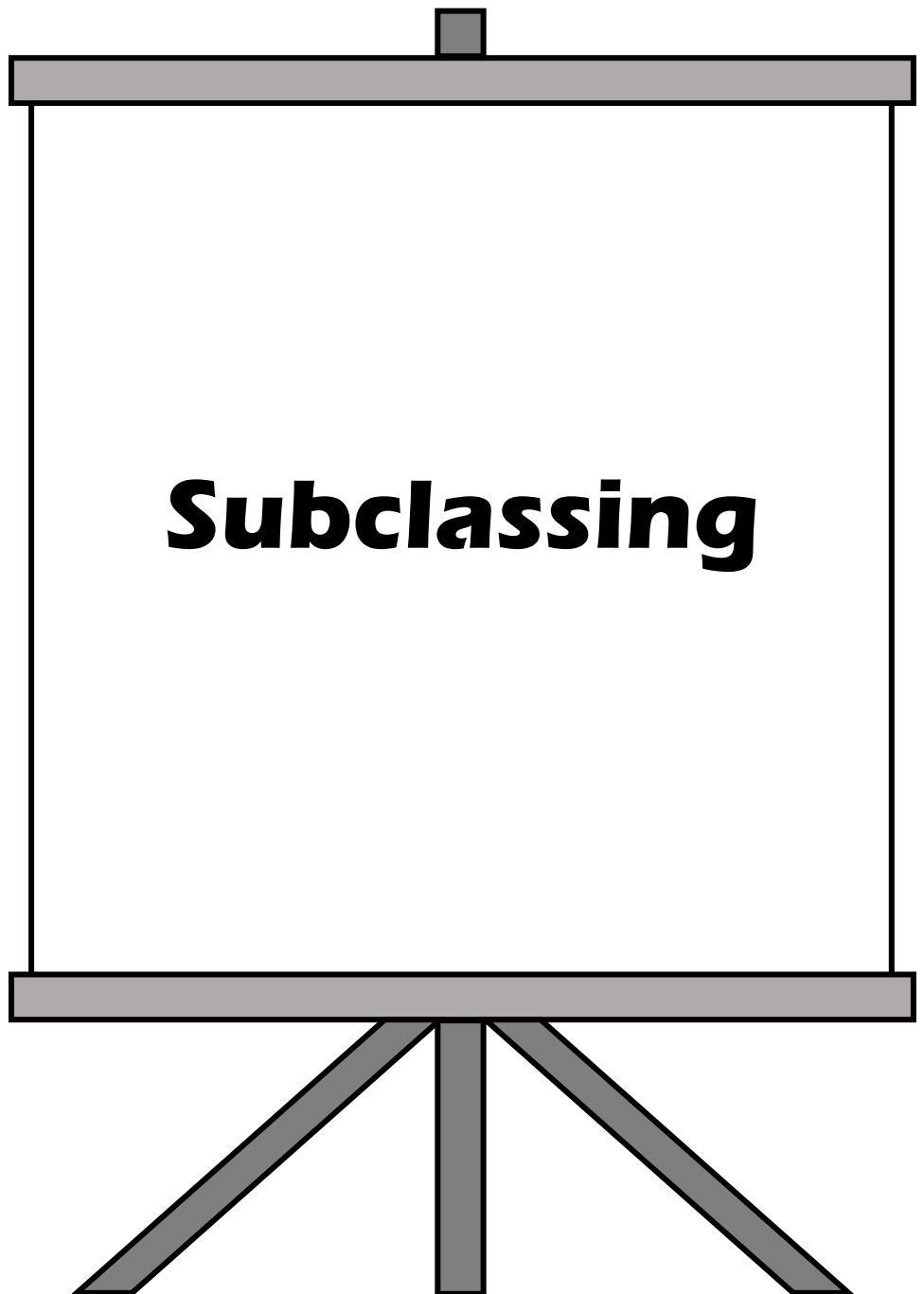
Output

Result of division: 5

4.2

LEARNING OUTCOMES

- ☑ Understanding of Inheritance: Subclassing facilitates the implementation of inheritance in object-oriented programming, enabling subclasses to inherit attributes and behaviors from their superclass. Learners gain insight into the hierarchical relationships between classes and the mechanisms through which properties and methods are inherited and extended.
- ☑ Encapsulation and Code Organization: Subclassing promotes encapsulation by allowing for the organization of code into hierarchical structures based on common functionalities and behaviors. Learners develop skills in modular design and code reuse, as they utilize superclass functionalities within subclasses, thereby enhancing code maintainability and scalability.



Definition

the creation of a new class (subclass) that inherits attributes and behaviors from an existing class (superclass), facilitating code reuse and specialization.



Preconditions: Preconditions are requirements that must be met before a method is executed. They define the state of the system or object that must be true for the method to function properly. By specifying preconditions, developers ensure that methods are invoked under suitable conditions, preventing unexpected behavior.



Sample Problem

Create a simple banking application that models different types of bank accounts using subclassing. Implement a superclass `BankAccount` with common functionalities and then create subclasses `SavingsAccount` and `CheckingAccount` with specific functionalities for each type of account.

```
// Superclass
class BankAccount {
    protected String accountNumber;
    protected double balance;

    public BankAccount(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited $" + amount + " into account " + accountNumber);
    }

    public void withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println("Withdrawn $" + amount + " from account " + accountNumber);
        } else {
            System.out.println("Insufficient balance in account " + accountNumber);
        }
    }

    public void displayBalance() {
        System.out.println("Account " + accountNumber + " balance: $" + balance);
    }
}
```



Output

Result of division: 5

```
// Subclass SavingsAccount
class SavingsAccount extends BankAccount {
    private double interestRate;

    public SavingsAccount(String accountNumber, double balance, double interestRate) { super(accountNumber, balance); this.interestRate = interestRate; }

    public void applyInterest() {
        double interest = balance * interestRate / 100;
        deposit(interest);
        System.out.println("Interest applied to account " + accountNumber + ": $" + interest);
    }
}

// Subclass CheckingAccount
class CheckingAccount extends BankAccount {
    private double overdraftLimit;

    public CheckingAccount(String accountNumber, double balance, double overdraftLimit) {
        super(accountNumber, balance); this.overdraftLimit = overdraftLimit;
    }

    @Override
    public void withdraw(double amount) {
        if (balance + overdraftLimit >= amount) {
            balance -= amount;
            System.out.println("Withdrawn $" + amount + " from account " + accountNumber);
        } else {
            System.out.println("Transaction declined: Insufficient funds and overdraft limit exceeded.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating instances of SavingsAccount and CheckingAccount
        SavingsAccount savingsAccount = new SavingsAccount("SA123", 1000, 2.5);
        CheckingAccount checkingAccount = new CheckingAccount("CA456", 500, 200);

        savingsAccount.deposit(200);
        savingsAccount.applyInterest(); savingsAccount.displayBalance();
        checkingAccount.withdraw(700); checkingAccount.displayBalance();
    }
}
```



NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES



Score

Learning Activity / Exercise # ____ Polymorphism

Name _____

Program-Year
Level _____

Date _____

Problem Statement:

Imagine you are developing a geometry calculator program for a mathematics education platform. The program should be capable of calculating the area of various shapes, including rectangles, circles, and triangles, based on user input. Each shape has its specific formula for calculating area. Your task is to design and implement a Java program that accurately computes the area of these shapes and displays the results to the user.

Requirements:

1. Implement a Java program that includes classes for representing different shapes: rectangles, circles, and triangles.
2. Each shape class should extend a common superclass called "Shape" and implement a method named "calculateArea()" to compute its area.
3. The "Rectangle" class should have attributes for length and width, the "Circle" class should have a radius attribute, and the "Triangle" class should have attributes for base and height.
4. Ensure that the "calculateArea()" method in each shape class correctly computes the area according to the specific formula for that shape.
5. In the main program, create instances of each shape class and calculate their respective areas.
6. Display the calculated areas of each shape to the user.

Your task is to implement the geometry calculator program following the given requirements, ensuring accurate area calculations for each shape.



Expected Output

Area of Rectangle: ____

Area of Circle: ____

Area of Triangle: ____



Score

Learning Activity / Exercise # ____
Polymorphism – Sample Source Code

Name

Program-Year
Level

Date

```
// Shape class representing a generic shape
abstract class Shape {
    // Abstract method to calculate area
    abstract double calculateArea();
}

// Rectangle class representing a rectangle shape
class Rectangle extends Shape {
    private double length;
    private double width;

    // Constructor to initialize length and width
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    // Implementation of calculateArea method for rectangle
    @Override
    double calculateArea() {
        return length * width;
    }
}

// Circle class representing a circle shape
class Circle extends Shape {
    private double radius;

    // Constructor to initialize radius
    public Circle(double radius) {
        this.radius = radius;
    }

    // Implementation of calculateArea method for circle
    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }
}
```

```
// Triangle class representing a triangle shape
class Triangle extends Shape {
    private double base;
    private double height;

    // Constructor to initialize base and height
    public Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    // Implementation of calculateArea method for triangle
    @Override
    double calculateArea() {
        return 0.5 * base * height;
    }
}

// Main class to demonstrate polymorphism
public class Main {
    public static void main(String[] args) {
        // Creating instances of different shapes
        Shape rectangle = new Rectangle(5, 4);
        Shape circle = new Circle(3);
        Shape triangle = new Triangle(4, 6);

        // Calculating and printing areas
        System.out.println("Area of Rectangle: " +
            rectangle.calculateArea());
        System.out.println("Area of Circle: " +
            circle.calculateArea());
        System.out.println("Area of Triangle: " +
            triangle.calculateArea());
    }
}
```

MODULE GOALS

- ❑ Problem Decomposition and Simplification: Abstraction enables developers to decompose complex problems into smaller, more manageable components represented by abstract classes, interfaces, and methods. By focusing on essential features and hiding unnecessary implementation details, abstraction simplifies problem-solving and promotes modular, reusable code design.
- ❑ Code Flexibility and Adaptability: Abstraction promotes code flexibility by decoupling the implementation details from the external interface. This allows developers to modify and extend the codebase more easily without affecting other parts of the system. By abstracting away implementation complexities, developers can adapt their code to changing requirements and new features more effectively, leading to more resilient and maintainable software systems.

College of Computing Information
Technologies

FLEX Course Material



Abstraction

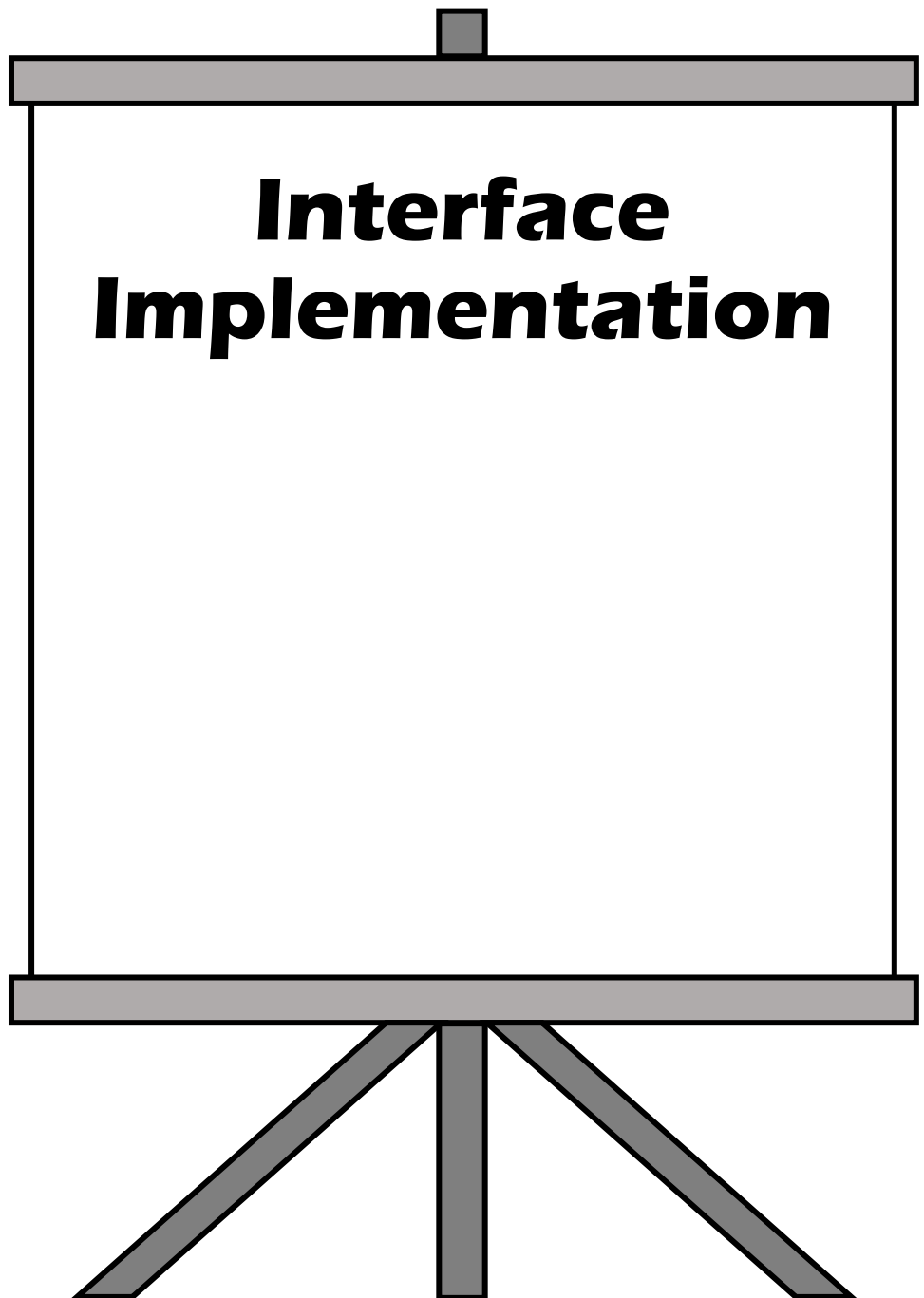
**Definition**

the process of hiding complex implementation details and showing only the essential features of an object, allowing for simplification and flexibility in software development

5.1

LEARNING OUTCOMES

- ❑ Understanding of Contractual Agreements: Interface inheritance helps learners grasp the concept of contractual agreements between classes. By implementing interfaces, developers commit to providing specific behaviors, fostering a deeper understanding of the importance of adhering to predefined contracts in software development.
- ❑ Enhanced Flexibility and Interchangeability: Through interface inheritance, learners gain insight into the flexibility and interchangeability of code. They understand how interfaces enable different classes to share common behaviors while allowing for diverse implementations. This fosters a mindset of modular design, promoting code that is more adaptable and easier to maintain.



Definition

Interface inheritance in Java enables subclasses to inherit method signatures from superinterfaces, mandating implementations for those methods, crucial for abstraction and contract definition, facilitating the creation of common interfaces for multiple classes, fostering polymorphism, and enabling code interchangeability while promoting loose coupling and modularity through standardized behavior, thus enhancing code reusability, maintainability, and extensibility, ultimately playing a pivotal role in promoting encapsulation, abstraction, and clear communication protocols in object-oriented programming.



Interface implementation in Java allows subclasses to inherit method signatures from superinterfaces, necessitating implementations for those methods, crucial for abstraction and contract definition, enabling the creation of common interfaces for multiple classes, thereby promoting polymorphism and code interchangeability, ultimately fostering effective collaboration, code reusability, and maintainability within object-oriented programming.



Sample Problem

```
// Interface
interface Shape { double calculateArea(); }

// Circle class implementing Shape interface
class Circle implements Shape { private double radius;
    public Circle(double radius) { this.radius = radius; }
    @Override
    public double calculateArea() { return Math.PI * radius * radius; }
}

// Rectangle class implementing Shape interface
class Rectangle implements Shape {
    private double length, width;
    public Rectangle(double length, double width) { this.length = length; this.width = width; }
    @Override
    public double calculateArea() { return length * width; }
}

public class Main {
    public static void main(String[] args) {
        // Interface inheritance
        Circle circle = new Circle(5);    System.out.println("Area of circle: " + circle.calculateArea());
        Rectangle rectangle = new Rectangle(4, 6);
        System.out.println("Area of rectangle: " + rectangle.calculateArea());
    }
}
```



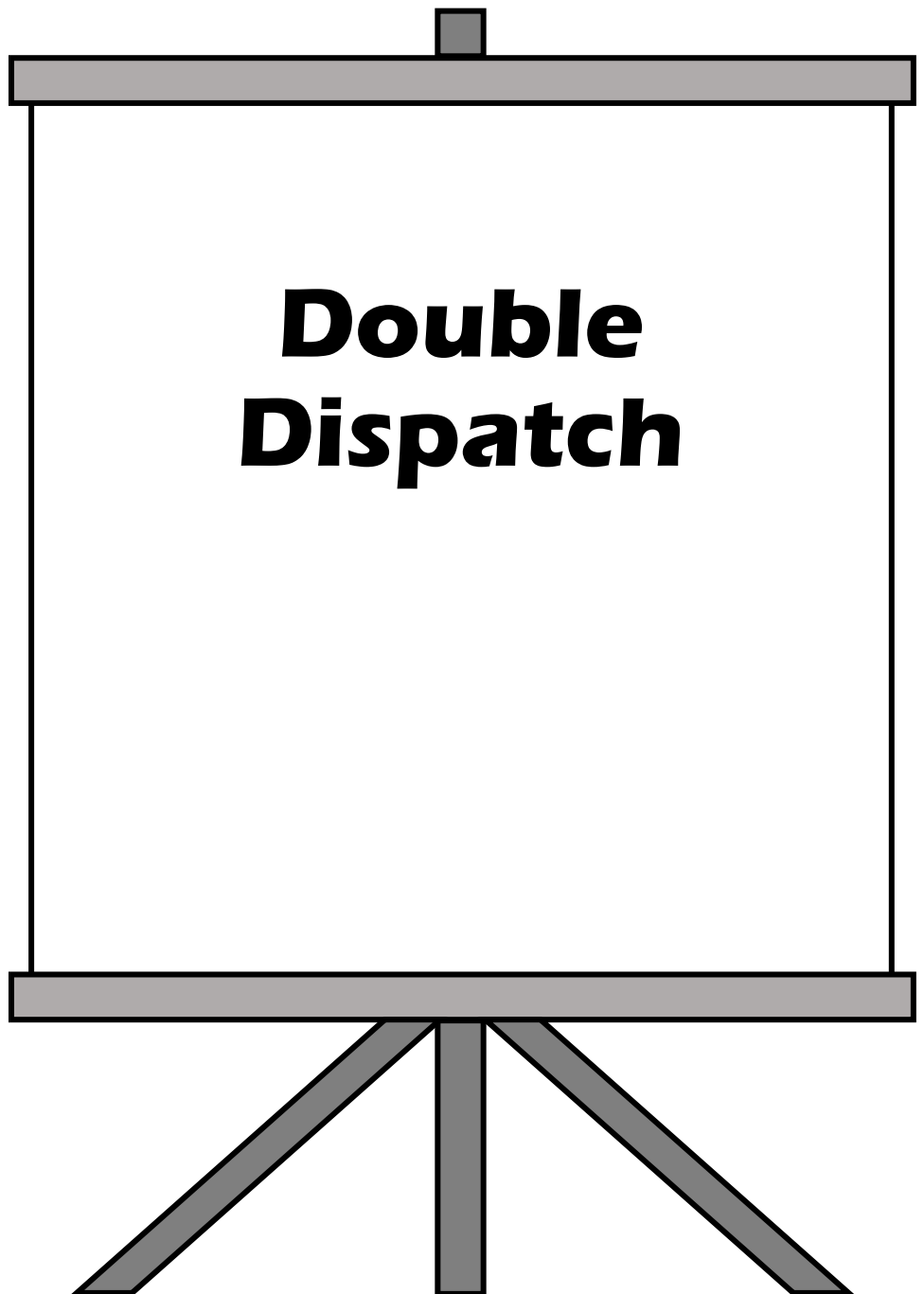
Output

```
Area of circle: 78.53981633974483
Area of rectangle: 24.0
```

5.2

LEARNING OUTCOMES

- ☑ Understanding Polymorphic Method Calls: Mastery of double dispatch involves understanding how polymorphic method calls work in Java, particularly when involving multiple types of objects and dynamic resolution based on runtime types. Learners should grasp the concept of method dispatch and how it relates to polymorphism and inheritance.
- ☑ Implementing Double Dispatch: Proficiency in double dispatch requires the ability to implement this design pattern effectively in Java code. This includes creating classes/interfaces with appropriate methods that utilize dynamic dispatch, understanding the interaction between different types of objects, and leveraging double dispatch to achieve flexible and extensible software designs.



Definition

a design pattern that resolves method calls dynamically based on the runtime types of two objects involved, facilitating polymorphic method calls between interacting objects.



Double dispatch is a design pattern used in object-oriented programming to handle polymorphic method calls involving multiple types of objects. It is a powerful technique that allows us to dynamically resolve method calls based on the runtime types of two objects rather than just one. In Java, double dispatch is particularly useful for achieving flexible and extensible code designs, especially in scenarios where multiple types of objects need to interact with each other.

- Double dispatch is a design pattern used to handle polymorphic method calls involving multiple types of objects in Java.
- It enables dynamic resolution of method calls based on the runtime types of two objects, facilitating flexible and extensible code designs.
- By implementing double dispatch, we can achieve loose coupling, improve maintainability, and handle complex interactions between objects more effectively in our Java applications.



Source Code

```
// Shape interface
interface Shape {
    void drawWith(Tool tool);
}

interface Tool { // Tool interface
    void useOnCircle(Circle circle);
    void useOnSquare(Square square);
}

class Pen implements Tool { // Concrete implementation of Tool interface
    @Override
    public void useOnCircle(Circle circle) { System.out.println("Using pen on circle"); }
    @Override
    public void useOnSquare(Square square) { System.out.println("Using pen on square"); }
}

class Circle implements Shape { // Circle class implementing Shape interface
    @Override
    public void drawWith(Tool tool) { tool.useOnCircle(this); // Double dispatch occurs here }
}

class Square implements Shape { // Square class implementing Shape interface
    @Override
    public void drawWith(Tool tool) {
        tool.useOnSquare(this); // Double dispatch occurs here
    }
}

public class Main { // Main class
    public static void main(String[] args) {
        Shape circle = new Circle(); Shape square = new Square();
        Tool pen = new Pen();
        circle.drawWith(pen); // Outputs: Using pen on circle
        square.drawWith(pen); // Outputs: Using pen on square
    }
}
```

In this example, we have two interfaces: Shape and Tool. The Shape interface defines a method drawWith(Tool tool) that accepts a Tool object as a parameter. The Tool interface defines methods useOnCircle(Circle circle) and useOnSquare(Square square).

The Circle and Square classes implement the Shape interface. Each of these classes overrides the drawWith() method and calls the appropriate method in the Tool interface based on the type of the shape.

The Pen class implements the Tool interface and provides concrete implementations for the useOnCircle() and useOnSquare() methods.

In the Main class, we create instances of Circle and Square, along with a Pen object. We then call the drawWith() method on each shape, passing the Pen object as a parameter. The double dispatch occurs when the drawWith() method of the shape delegates the call to the appropriate method in the Tool interface based on the runtime type of the shape object, and then based on the runtime type of the tool object.



NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES



Score

Learning Activity / Exercise # ____ Double Dispatch

Name

Program-Year
Level

Date

You are tasked with developing a shape drawing application that allows users to draw various shapes using different drawing tools. The application should support drawing circles and squares, and users should be able to select between different drawing tools such as a pen or a brush.

Requirements:

1. Define a set of interfaces and classes to represent shapes and drawing tools in the application.
2. Implement the Shape interface to define methods for drawing shapes with different tools.
3. Implement the Tool interface to define methods for drawing circles and squares with different drawing tools.
4. Create concrete implementations of drawing tools, such as a Pen and a Brush, each capable of drawing circles and squares.
5. Ensure that the drawing operations use double dispatch to dynamically select the appropriate drawing method based on the type of shape and drawing tool.
6. Develop a Main class to demonstrate the functionality of the application. Instantiate shapes (circles and squares) and drawing tools (pen and brush), and use the `drawWith()` method to draw shapes with different tools.

Constraints:

- Ensure that the implementation follows the principles of abstraction and encapsulation.
- Use appropriate design patterns, such as double dispatch, to achieve flexibility and maintainability in the codebase.
- The application should be well-structured, modular, and easy to extend with additional shapes and drawing tools in the future.

Note: Adjust the problem statement as per your specific requirements and constraints.



Expected Output

- When drawing a circle with a pen: "Drawing circle with pen"
- When drawing a circle with a brush: "Drawing circle with brush"
- When drawing a square with a pen: "Drawing square with pen"
- When drawing a square with a brush: "Drawing square with brush"

MODULE GOALS

- ▣ Precise Specification and Analysis: Formal models of OOP systems provide a rigorous framework for specifying the structure, behavior, and properties of software systems using mathematical notations and formal semantics. By learning to use formal models such as Object-Z, Unified Modeling Language (UML), or algebraic specification languages, developers can gain skills in precisely specifying system requirements and analyzing system designs to ensure correctness, consistency, and completeness.
- ▣ Improved System Understanding and Design: Studying formal models of OOP systems enhances developers' understanding of object-oriented concepts and principles, such as encapsulation, inheritance, polymorphism, and message passing. Through the use of formal notations and modeling techniques, developers can gain insights into the relationships between system components, interactions between objects, and the dynamic behavior of the system.

College of Computing Information
Technologies

FLEX Course Material

**Formal Models in
OOP****Definition**

provide a rigorous framework for precise specification, analysis, and verification of system requirements and designs, ensuring correctness, consistency, and reliability of software systems.



Formal models in Object-Oriented Programming (OOP) are mathematical representations used to precisely specify the structure, behavior, and properties of software systems. These models typically incorporate formal notations, such as mathematical logic, set theory, or algebraic specification languages, to define classes, objects, inheritance, and interactions between components. By providing a formal framework for analysis and verification, formal models aid in ensuring correctness, consistency, and reliability of OOP designs and implementations.



Object-Z

an extension of the Z notation, a formal specification language based on set theory and first-order predicate logic. Object-Z extends Z with constructs for modeling object-oriented concepts such as classes, objects, inheritance, and encapsulation. It provides a formal notation for specifying the structure and behavior of object-oriented systems.



Unified Modeling Language (UML)

primarily a graphical notation for modeling software systems, it also provides a formal foundation for specifying the structure, behavior, and interactions of object-oriented systems. UML includes formal semantics for its various diagrams, such as class diagrams, sequence diagrams, and state diagrams, allowing for precise specification and analysis of OOP systems



Object-Oriented Petri Nets (OOPNs)

a mathematical modeling technique used for describing concurrent systems. Object-Oriented Petri Nets extend traditional Petri Nets to incorporate object-oriented concepts such as classes, objects, and message passing.



Algebraic Specification Languages

provide formal frameworks for specifying abstract data types, operations, and their properties using algebraic equations and axioms. While not specific to object-oriented modeling, these languages can be used to formally specify the abstract structure and behavior of object-oriented systems.



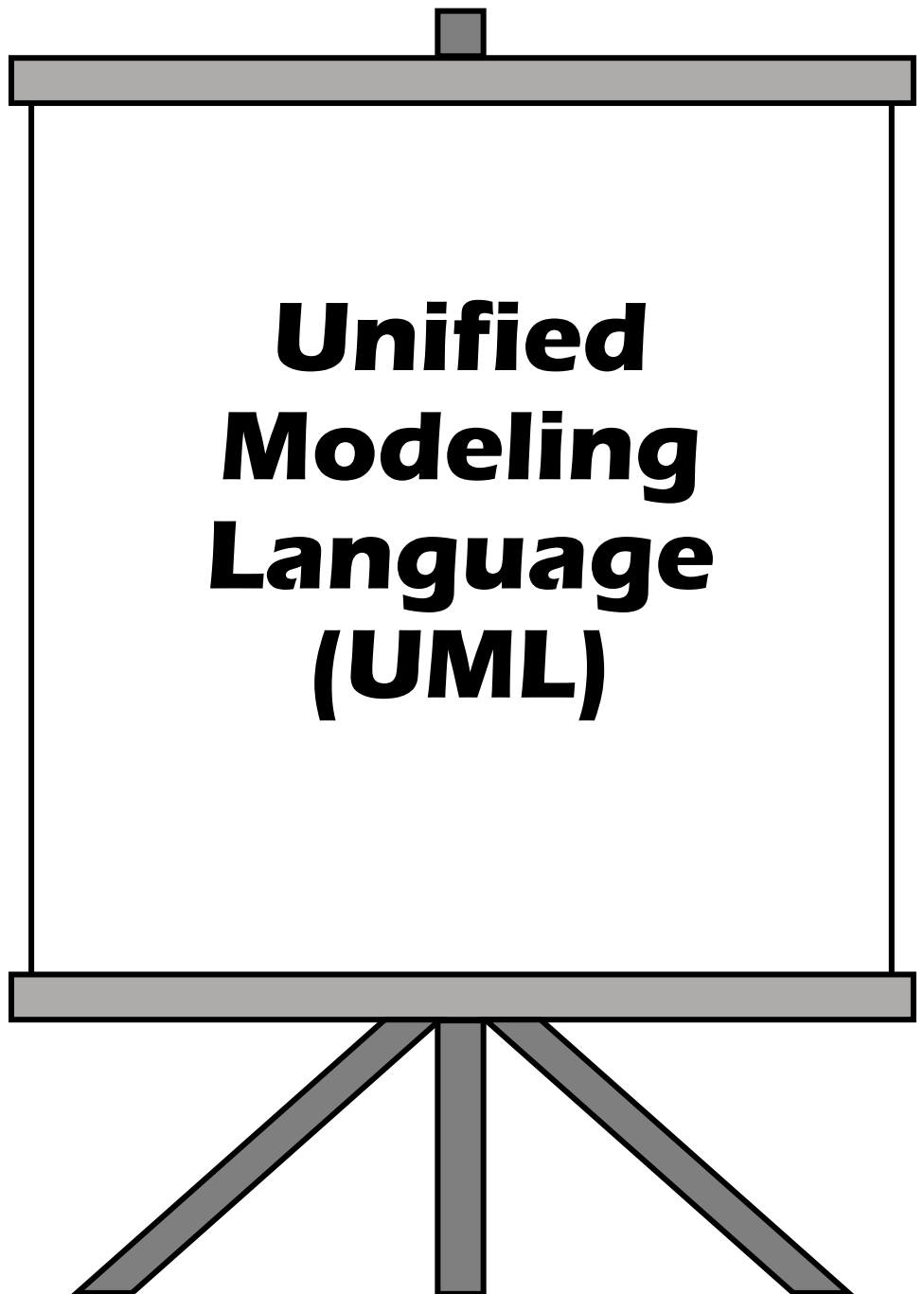
Zermelo–Fraenkel Set Theory with Classes (ZF + C)

an extension of Zermelo–Fraenkel set theory (ZF) that incorporates classes as first-class citizens. It provides a formal foundation for modeling object-oriented concepts such as classes, objects, and inheritance within the framework of set theory, allowing for rigorous analysis and reasoning about object-oriented systems.

6.1

LEARNING OUTCOMES

- ❑ Improved Communication and Collaboration: Learning UML enhances communication and collaboration among stakeholders, including developers, designers, and clients, by providing a standardized visual language for expressing system designs and requirements. Through UML diagrams such as class diagrams, sequence diagrams, and use case diagrams, stakeholders can better understand, discuss, and refine system architectures and functionalities.
- ❑ Enhanced System Understanding and Design: Mastering UML enables developers to gain a deeper understanding of complex software systems and their interactions. By creating and analyzing UML diagrams, developers can visualize the structure, behavior, and relationships between system components, facilitating better system design decisions and leading to more robust, scalable, and maintainable software solutions.



Definition

a standardized visual language used in software engineering to depict, specify, design, and document the structure and behavior of complex systems



UML (Unified Modeling Language) is a standardized visual language used in software engineering to depict, specify, design, and document the structure and behavior of complex systems. It provides a set of graphical notations for representing system components, their relationships, and interactions, facilitating communication and understanding among stakeholders. UML diagrams include class diagrams, use case diagrams, sequence diagrams, and others, each serving different purposes in modeling software systems.



Class Diagram

represents the static structure of a system, showing classes, attributes, methods, and their relationships. It illustrates the entities in the system and their associations.



Use Case Diagram

illustrates the functional requirements of a system by depicting actors, use cases, and their relationships. It shows how users interact with the system to achieve specific goals.



Sequence Diagram

a mathematical modeling technique used for describing concurrent systems. Object-Oriented Petri Nets extend traditional Petri Nets to incorporate object-oriented concepts such as classes, objects, and message passing.



Activity Diagram

provide formal frameworks for specifying abstract data types, operations, and their properties using algebraic equations and axioms. While not specific to object-oriented modeling, these languages can be used to formally specify the abstract structure and behavior of object-oriented systems.



State Diagram

models the behavior of a single object or system component over its lifecycle, showing different states and transitions between them. It illustrates how the object responds to events and changes its state.



Component Diagram

depicts the physical components of a system and their dependencies, showing how system elements are organized and interact at runtime



Deployment Diagram

illustrates the physical deployment of software components on hardware nodes, showing the distribution of components across different nodes in a system architecture.



A **class diagram** is a type of UML (Unified Modeling Language) diagram that represents the static structure of a system by depicting classes, their attributes, methods, relationships, and constraints. It illustrates the various entities in the system and how they are related to each other. Class diagrams are widely used in software engineering for modeling the structure of object-oriented systems, providing a visual representation of the classes and their interactions.

To create a class diagram, follow these steps:

1. **Identify Classes:** Identify the classes in your system based on its requirements and functionalities. Each class should represent a distinct entity or concept within the system.
2. **Determine Attributes:** Determine the attributes (data members or properties) for each class. Attributes represent the state of the class and are listed within the class rectangle.
3. **Define Methods:** Define the methods (functions or operations) for each class. Methods represent the behavior or operations that the class can perform and are also listed within the class rectangle.
4. **Establish Relationships:** Identify and establish relationships between classes. Common types of relationships include association, aggregation, composition, inheritance, and dependency. Use appropriate connectors to represent these relationships between classes.
5. **Add Multiplicity:** Specify the multiplicity of the relationships, indicating how many instances of one class are associated with instances of another class. Multiplicity is represented using numbers or asterisks near the ends of association lines.
6. **Include Visibility Modifiers:** Optionally, include visibility modifiers (such as public, private, and protected) to specify the accessibility of attributes and methods within classes.
7. **Add Constraints (Optional):** If needed, include constraints or additional rules that must be satisfied by the classes and their relationships.

```
+-----+
|  BankAccount  |
+-----+
| - accountNumber : String |
| - balance : double  |
+-----+
| + deposit(amount: double) : void |
| + withdraw(amount: double) : void |
| + getBalance() : double |
+-----+

+-----+
|  BankEmployee  |
+-----+
| - employeeId : int |
| - name : String  |
| - position : String |
+-----+
| + BankEmployee(name: String, position: String) |
| + getEmployeeId() : int |
| + getName() : String |
| + getPosition() : String |
+-----+
```

Two classes (BankAccount and BankEmployee) are identified.

Each class has its attributes and methods listed within its rectangle.

Relationships between classes are not shown in this example, but you could establish associations between BankAccount and Customer, and between BankAccount and BankEmployee, to represent the ownership and management relationships, respectively.



A **use case diagram** is a type of UML (Unified Modeling Language) diagram that provides a graphical representation of the functional requirements of a system. It depicts the interactions between users (actors) and the system to achieve specific goals or perform certain tasks. Use case diagrams help to visualize the system's behavior from the perspective of its users, illustrating how they interact with the system to accomplish various functionalities.

To create a Use Case diagram, follow these steps:

- **Identify Actors:** Identify the actors involved in the system. Actors are external entities (such as users or systems) that interact with the system to perform specific tasks or achieve goals.
- **Identify Use Cases:** Identify the use cases of the system. Use cases represent specific functionalities or tasks that the system performs to satisfy the needs of its users or actors. Each use case should correspond to a specific user goal or requirement.
- **Draw Actors and Use Cases:** Draw actors as stick figures (representing human users) or simple shapes (representing external systems) on the left side of the diagram. Draw use cases as ovals or ellipses in the middle or right side of the diagram.
- **Connect Actors and Use Cases:** Use solid lines with arrows to connect actors to the use cases they are involved in. Each arrow represents the direction of the interaction, indicating that the actor initiates or participates in the use case.
- **Include Relationships:** Optionally, include relationships between use cases, such as "extends" and "includes" relationships, to represent common functionalities shared among multiple use cases or alternative flows within a use case.
- **Add System Boundary (Optional):** Optionally, draw a rectangle around the use cases to represent the system boundary. This helps to visually distinguish the system from its external actors.
- **Label Use Cases:** Label each use case with a descriptive name that clearly defines the functionality it represents. Use concise and meaningful names to ensure clarity and understanding.
- **Include Actors Descriptions (Optional):** Optionally, include brief descriptions or roles for each actor to provide additional context and understanding of their roles and responsibilities in the system.



The actors are Customer and Administrator.

The use cases are Browse Products, Add to Cart, Checkout, and Make Payment.

The Customer actor interacts with the use cases Browse Products, Add to Cart, Checkout, and Make Payment.

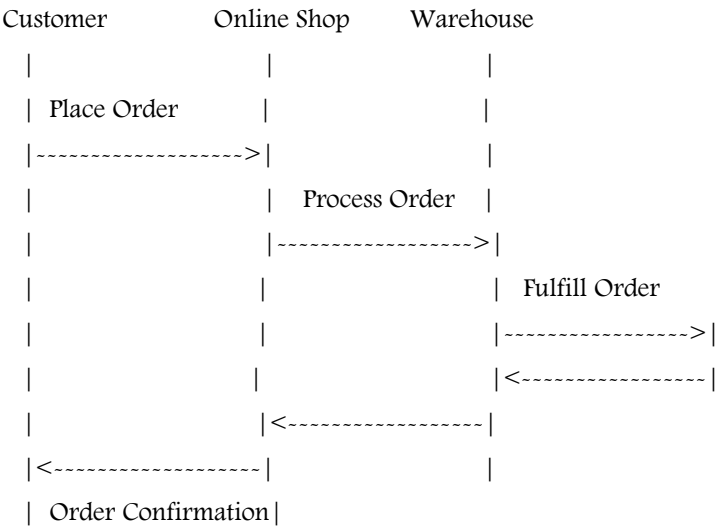
The Administrator actor may interact with the system for administrative tasks not explicitly shown in this diagram.



A **sequence diagram** is a type of UML (Unified Modeling Language) diagram that illustrates the interactions between objects or components over time in a system. It depicts the sequence of messages exchanged between objects, along with the lifelines representing the lifespan of each object during the interaction. Sequence diagrams provide a dynamic view of system behavior, showing the flow of control and communication between objects in response to specific events or scenarios.

To create a sequence diagram, follow these steps:

1. **Identify Objects:** Identify the objects or components involved in the interaction. These objects represent the system components, modules, or entities that participate in the sequence of messages.
2. **Determine Lifelines:** Create lifelines for each identified object. Lifelines represent the lifespan of objects during the interaction and are depicted as vertical dashed lines.
3. **Sequence Messages:** Identify the sequence of messages exchanged between objects. Messages represent the communication between objects and can be synchronous (indicated by solid arrows) or asynchronous (indicated by dashed arrows).
4. **Order Messages:** Order the messages chronologically according to the sequence of events in the interaction. Messages are placed along the lifelines to show when they occur during the interaction.
5. **Include Activation Bars (Optional):** Optionally, include activation bars to represent the period during which an object is active and processing a message. Activation bars are drawn on lifelines and indicate the duration of object execution.
6. **Label Messages:** Label each message with a description that explains the purpose or content of the message. Use concise and meaningful labels to clearly convey the message's intent.
7. **Include Return Messages (Optional):** Optionally, include return messages to indicate the response or result of a previously sent message. Return messages are depicted with dashed arrows and indicate the flow of control back to the sender.
8. **Add Comments (Optional):** Optionally, include comments or annotations to provide additional explanations or clarifications about specific aspects of the sequence diagram.
9. **Review and Validate:** Review the sequence diagram to ensure that it accurately represents the intended sequence of interactions and communication between objects. Validate the diagram against the system requirements and use cases to confirm its correctness.
10. **Finalize and Share:** Once validated, finalize the sequence diagram and share it with stakeholders, such as developers, designers, and clients, to communicate the system's behavior and interactions effectively.



Lifelines are represented by vertical dashed lines for each object (Customer, Online Shop, Warehouse).

Messages are depicted as arrows between lifelines, showing the flow of communication (Place Order, Process Order, Fulfill Order, Order Confirmation).

Activation bars are not included in this simple example, but they would represent the period during which an object is active and processing a message.

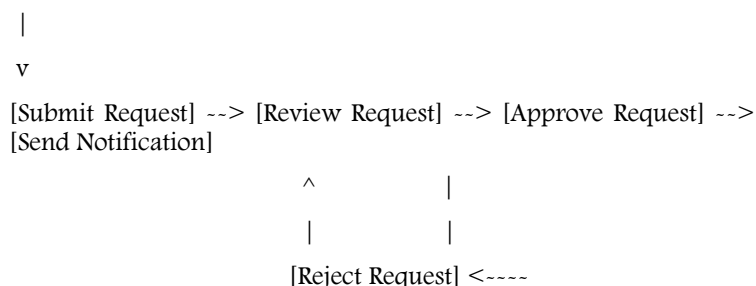


An **activity diagram** is a type of UML (Unified Modeling Language) diagram that represents the flow of activities or processes within a system. It depicts the sequence of actions, decisions, and control flows that occur during the execution of a specific functionality or business process. Activity diagrams provide a visual representation of the workflow, showing how activities are ordered, controlled, and interconnected. They are commonly used during the analysis and design phases of software development to model business processes, system workflows, and use case scenarios.

To create a activity diagram, follow these steps:

1. **Identify Activities:** Recognize the tasks involved, representing specific actions contributing to the diagram's overall goal.
2. **Determine Start and End Points:** Define the initiation and completion of the activity diagram.
3. **Draw Initial and Final Nodes:** Depict the start and end points as solid circles, with the final node surrounded by a hollow circle.
4. **Connect Activities:** Use arrows to connect activities, showing the flow of control between them.
5. **Include Decision Points:** Optionally, add decision points as diamonds to indicate branching or conditional behavior.
6. **Add Merge Points:** Optionally, incorporate merge points to symbolize the merging of control flows.
7. **Include Fork and Join Nodes:** Optionally, utilize fork nodes to split a flow into multiple concurrent flows and join nodes to synchronize them back into a single flow.
8. **Label Activities and Nodes:** Clearly label each element to clarify its purpose or functionality.
9. **Include Guard Conditions:** Optionally, specify conditions for transitions leaving decision points.
10. **Review and Validate:** Ensure the accuracy and completeness of the diagram, validating it against system requirements and use cases.
11. **Finalize and Share:** Once validated, finalize the diagram and share it with stakeholders for effective communication of the process or workflow.

(Start)



In this example:

Activities are represented as rectangles (Submit Request, Review Request, Approve Request, Send Notification, Reject Request).

Decision points are represented as diamonds (Review Request, Approve Request, Reject Request).

Transitions between activities are depicted as arrows, showing the flow of control (Submit Request leads to Review Request, Review Request leads to either Approve Request or Reject Request, etc.).

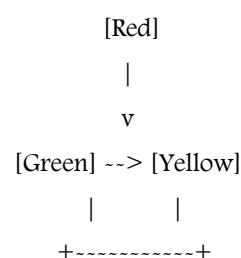
Initial and final nodes are not explicitly shown in this example, but they would represent the start and end points of the activity diagram.



A **state/state machine/statechart diagram**, represents the dynamic behavior of a system by depicting the states, transitions, and events that an object or system undergoes over time. It illustrates the various states that an object can be in and the transitions between these states in response to external events or stimuli. State diagrams are particularly useful for modeling the behavior of complex systems, such as software applications or control systems, where objects exhibit different behaviors depending on their internal state and external inputs.

To create a state diagram, follow these steps:

1. **Identify States:** Identify the possible states that an object or system can be in. These states represent different conditions or modes of operation that the object can exhibit.
2. **Determine Events:** Determine the events or stimuli that can trigger transitions between states. Events represent external inputs, actions, or conditions that cause the object to change its state.
3. **Draw States:** Draw circles or rectangles to represent each state identified in step 1. Label each state with a descriptive name that clearly defines its meaning or purpose.
4. **Draw Transitions:** Draw arrows between states to represent transitions triggered by events. Label each transition arrow with the name of the event that causes the transition.
5. **Include Initial and Final States (Optional):** Optionally, include initial and final states to represent the starting and ending points of the state diagram. An initial state is depicted as a filled circle with an incoming arrow, while a final state is represented as a circle with double borders.
6. **Add Guard Conditions (Optional):** Optionally, include guard conditions on transitions to specify additional criteria that must be met for the transition to occur. Guard conditions are depicted as Boolean expressions next to transition arrows.
7. **Include Actions (Optional):** Optionally, include actions or activities associated with transitions to indicate the behavior or tasks performed when transitioning between states. Actions are typically depicted as labels next to transition arrows.
8. **Review and Validate:** Review the state diagram to ensure that it accurately represents the behavior of the system or object. Validate the diagram against system requirements and use cases to confirm its correctness and completeness.
9. **Finalize and Share:** Once validated, finalize the state diagram and share it with stakeholders, such as developers, designers, and clients, to communicate the dynamic behavior of the system effectively.



In this example:

States are represented as rectangles (Red, Green, Yellow).

Transitions are depicted as arrows between states, indicating the flow of control (Red transitions to Green, Green transitions to Yellow, Yellow transitions back to Red).

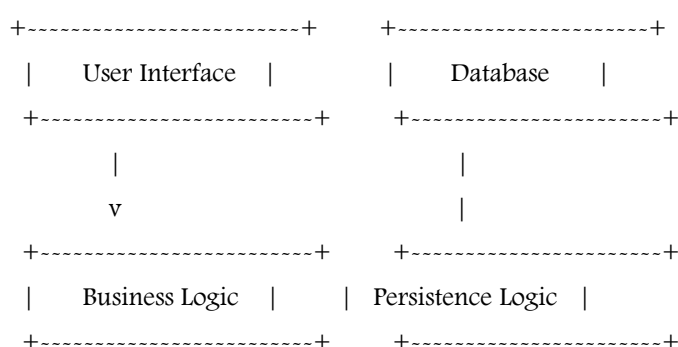
Initial and final states are not explicitly shown in this example, but they would represent the starting and ending points of the state diagram.



A **component diagram** visually represents a system's physical components and their interdependencies, offering insight into the system's structure and runtime interactions. Components symbolize modular elements like executable files or classes, while dependencies indicate how components rely on one another to meet system needs. Widely utilized during software design, these diagrams facilitate system decomposition and modularization, aiding in architectural visualization and planning.

To create a component diagram, follow these steps:

1. **Identify Components:** Identify the physical components of the system. These can include executable files, libraries, classes, modules, or subsystems.
2. **Determine Relationships:** Determine the dependencies and relationships between the components. These can include associations, dependencies, aggregations, compositions, or generalizations.
3. **Draw Components:** Draw rectangles or squares to represent each component identified in step 1. Label each component with a descriptive name that clearly defines its purpose or functionality.
4. **Connect Components:** Connect the components using lines or arrows to represent the relationships between them. Use different types of connectors (such as association arrows or dependency lines) to denote the nature of the relationships.
5. **Include Interfaces (Optional):** Optionally, include interfaces to represent the contracts or APIs provided by the components. Interfaces are depicted as circles or rectangles with the interface name.
6. **Add Ports (Optional):** Optionally, include ports to represent the connection points or interfaces through which components interact. Ports are depicted as small squares or circles on the edges of component rectangles.
7. **Include Dependencies (Optional):** Optionally, include dependencies to represent the relationships where one component relies on another for functionality but is not part of its structure. Dependencies are depicted as dashed arrows.
8. **Include Annotations (Optional):** Optionally, include annotations or notes to provide additional information or clarify the purpose of specific components or relationships.
9. **Review and Validate:** Review the component diagram to ensure that it accurately represents the system's physical components and their relationships. Validate the diagram against system requirements and architecture documentation to confirm its correctness and completeness.
10. **Finalize and Share:** Once validated, finalize the component diagram and share it with stakeholders, such as developers, designers, and clients, to communicate the system architecture effectively.



In this example:

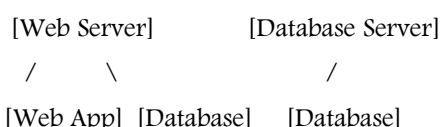
- Components are represented as rectangles (User Interface, Business Logic, Database, Persistence Logic).
- Relationships between components are depicted using lines or arrows, indicating how components interact (User Interface depends on Business Logic, Business Logic depends on Database, etc.).



A **deployment diagram** visualizes the physical distribution of software components onto hardware nodes within a system, showcasing the arrangement of executable files, libraries, or modules across servers, computers, or devices in a network. It illustrates the relationships and dependencies between these components and hardware nodes, elucidating their interaction and communication to facilitate system functionality.

To create a deployment diagram, follow these steps:

1. **Identify Software Components and Hardware Nodes:** Identify the software components (e.g., executable files, libraries, modules) and hardware nodes (e.g., servers, computers, devices) involved in the system.
2. **Determine Relationships and Dependencies:** Determine the relationships and dependencies between software components and hardware nodes. Understand how components are deployed onto nodes and how they interact with each other.
3. **Draw Nodes:** Draw rectangles or squares to represent each hardware node identified in step 1. Label each node with a descriptive name, such as "Server", "Desktop Computer", or "Mobile Device".
4. **Draw Components:** Draw nodes inside each hardware node to represent the software components deployed on that node. Use rectangles or squares to represent components and label each component with its name or identifier.
5. **Connect Nodes and Components:** Connect components to the nodes on which they are deployed using lines or arrows. Use different types of connectors (such as association arrows or dependency lines) to denote the nature of the relationship between components and nodes.
6. **Include Interfaces (Optional):** Optionally, include interfaces to represent the communication channels or connections between components deployed on different nodes. Interfaces are depicted as circles or rectangles with the interface name.
7. **Add Communication Paths (Optional):** Optionally, add communication paths to represent the network connections or communication channels between hardware nodes. Use lines or arrows to depict communication paths and label each path with a descriptive name.
8. **Include Dependencies (Optional):** Optionally, include dependencies to represent the relationships where one component relies on another for functionality but is not part of its structure. Dependencies are depicted as dashed arrows.
9. **Include Annotations (Optional):** Optionally, include annotations or notes to provide additional information or clarify the purpose of specific components, nodes, or relationships.
10. **Review and Validate:** Review the deployment diagram to ensure that it accurately represents the physical deployment of software components onto hardware nodes. Validate the diagram against system requirements and architecture documentation to confirm its correctness and completeness.



In this example:

- Hardware nodes are represented as rectangles (Web Server, Database Server).
- Software components are depicted as rectangles inside the hardware nodes (Web App, Database).
- Communication paths are not explicitly shown in this example but would represent the network connections between hardware nodes.



NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

Score

Learning Activity / Exercise # ____
UML Diagram

Name

Program-Year
Level

Date

As part of the MRT station project, you must be able to create comprehensive UML diagrams that depict the various processes and functionalities of the MRT station system. This includes creating

1. Use case diagrams to illustrate the interactions between users (e.g., passengers, staff) and the system,
2. Class diagrams to model the structure and relationships of the system's components (e.g., ticketing machines, platforms, trains),
3. Sequence diagrams to showcase the flow of interactions between objects during specific scenarios (e.g., passenger boarding process), and
4. Activity diagrams to visualize the workflow and activities within the system (e.g., ticket purchasing process).

These UML diagrams will serve as essential documentation for understanding, designing, and implementing the MRT station system effectively

MODULE GOALS

- ▣ Understanding of Common Design Solutions: Learning design patterns equips individuals with knowledge of common solutions to recurring design problems. This enables them to recognize and apply established solutions, reducing the need to reinvent the wheel and promoting efficient and effective software development.
- ▣ Enhanced Software Design Skills: Studying design patterns enhances one's ability to design flexible, reusable, and maintainable software systems. By understanding various design patterns and their appropriate application contexts, individuals can make informed design decisions and create robust software architectures that are adaptable to change and scalable over time.

College of Computing Information
Technologies

FLEX Course Material



Design Patterns

**Definition**

reusable solutions to common software design problems that help developers address recurring challenges efficiently and effectively



Formal models in Object-Oriented Programming (OOP) are mathematical representations used to precisely specify the structure, behavior, and properties of software systems. These models typically incorporate formal notations, such as mathematical logic, set theory, or algebraic specification languages, to define classes, objects, inheritance, and interactions between components. By providing a formal framework for analysis and verification, formal models aid in ensuring correctness, consistency, and reliability of OOP designs and implementations.



Creational Patterns

These patterns focus on the process of object creation, providing flexible ways to create objects while hiding the creation logic and making the system independent of how its objects are created, composed, and represented. Examples include Singleton, Factory Method, Abstract Factory, Builder, Prototype.



Structural Patterns

These patterns deal with the composition of classes or objects, focusing on the relationship between classes or objects to form larger structures. They help ensure that if one part of a system changes, the entire system doesn't need to do the same. Examples include Adapter, Bridge, Composite, Decorator, Facade, Proxy.



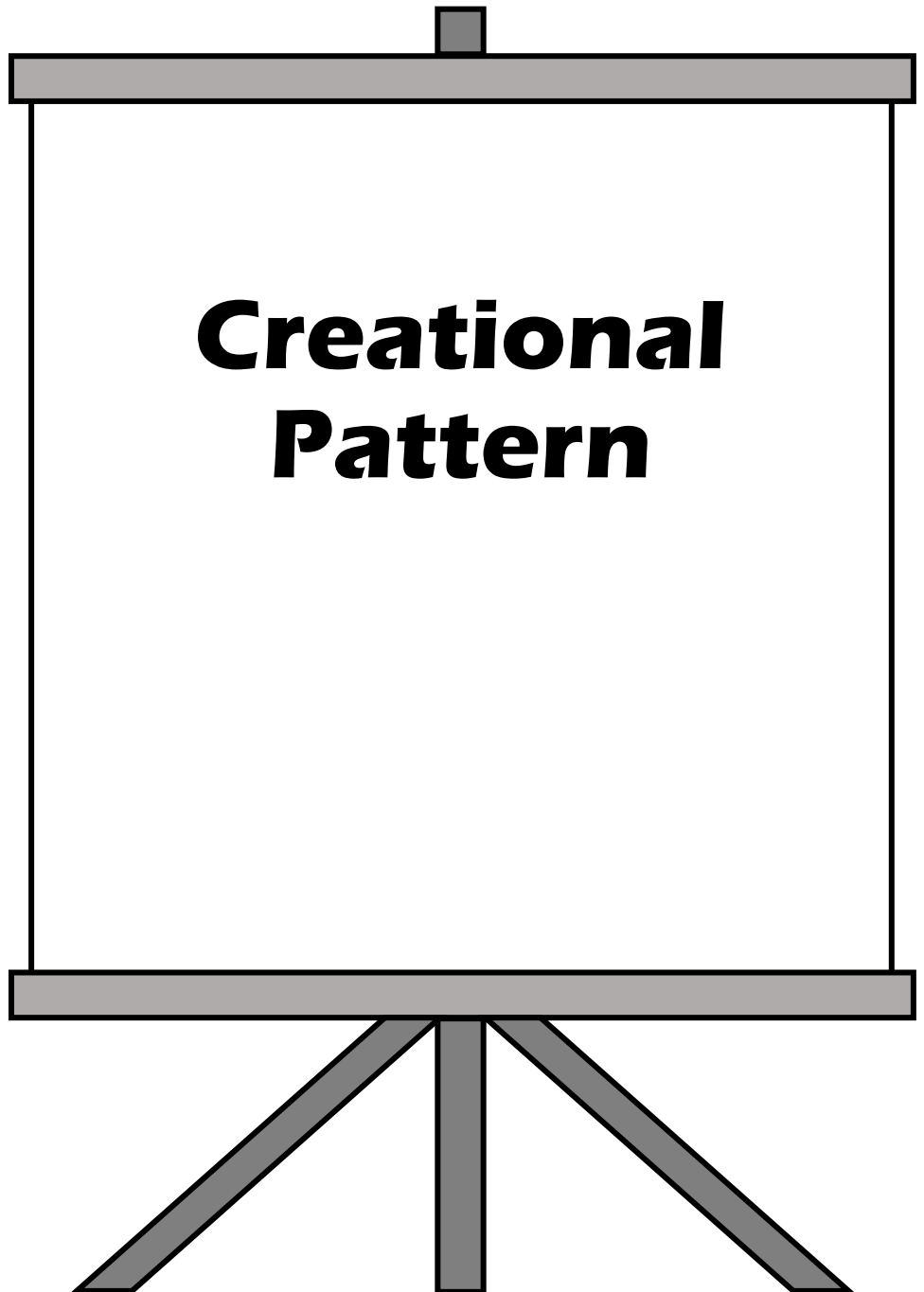
Behavioral Patterns

These patterns focus on the interaction between objects and the responsibility of objects, how they communicate, and how they collaborate to complete a task. They help in defining how objects interact in a loosely coupled way, promoting flexibility and maintainability. Examples include Observer, Strategy, Template Method, Command, State, Iterator.

7.1

LEARNING OUTCOMES

- ❑ Understanding of Object Creation Mechanisms: Learning creational patterns equips individuals with knowledge of various object creation mechanisms, including Singleton, Factory Method, Abstract Factory, Builder, and Prototype patterns. This understanding helps developers choose the appropriate pattern to instantiate objects based on different scenarios and requirements.
- ❑ Ability to Design Flexible and Efficient Object Creation Processes: Studying creational patterns enhances one's ability to design flexible and efficient object creation processes. By understanding how to abstract the instantiation process and decouple the client code from the object creation logic, developers can create systems that are more maintainable, scalable, and adaptable to change.



Definition

provide solutions for object creation, focusing on flexible instantiation mechanisms while promoting code reuse and maintainability



Creational patterns are a category of design patterns that deal with object creation mechanisms, trying to create objects in a manner suitable to the situation. They aim to abstract the instantiation process, making the system independent of how objects are created, composed, and represented. Creational patterns provide flexibility in the creation of objects and help in minimizing the complexity of the code by controlling the creation process.



Singleton Pattern

The Singleton pattern ensures that a class has only one instance and provides a global point of access to that instance.

This pattern is useful when you need to restrict the instantiation of a class to only one object, such as when managing shared resources or controlling access to a resource.

Example: Managing a database connection pool in a multi-threaded application where only one connection instance is needed.



Abstract Factory Pattern

The Abstract Factory pattern provides an interface for creating families of related or dependent objects without specifying their concrete classes.

It allows a client to create objects without needing to know the specifics of how they're created, promoting flexibility and modularity in the design.

Example: A UI toolkit providing abstract factories for creating different types of GUI components like buttons, text fields, and checkboxes, with concrete implementations for different operating systems.



Prototype Pattern

The Prototype pattern creates new objects by cloning an existing instance, avoiding the need for subclassing to create new objects.

It allows the creation of new objects by copying an existing object's state, providing a way to produce new instances more efficiently and with less overhead.

Example: Creating new instances of complex objects like graphical shapes by cloning a prototype shape object, rather than creating them from scratch.



Factory Method Pattern

The Factory Method pattern defines an interface for creating objects but allows subclasses to alter the type of objects that will be created.

It provides a way to encapsulate object creation logic and delegate it to subclasses, promoting loose coupling between the creator and the products it creates.

Example: A framework providing an abstract class with a method for creating document objects, while concrete subclasses implement this method to create specific types of documents like PDFs or Word documents.



Builder Pattern

The Builder pattern separates the construction of a complex object from its representation, allowing the same construction process to create different representations.

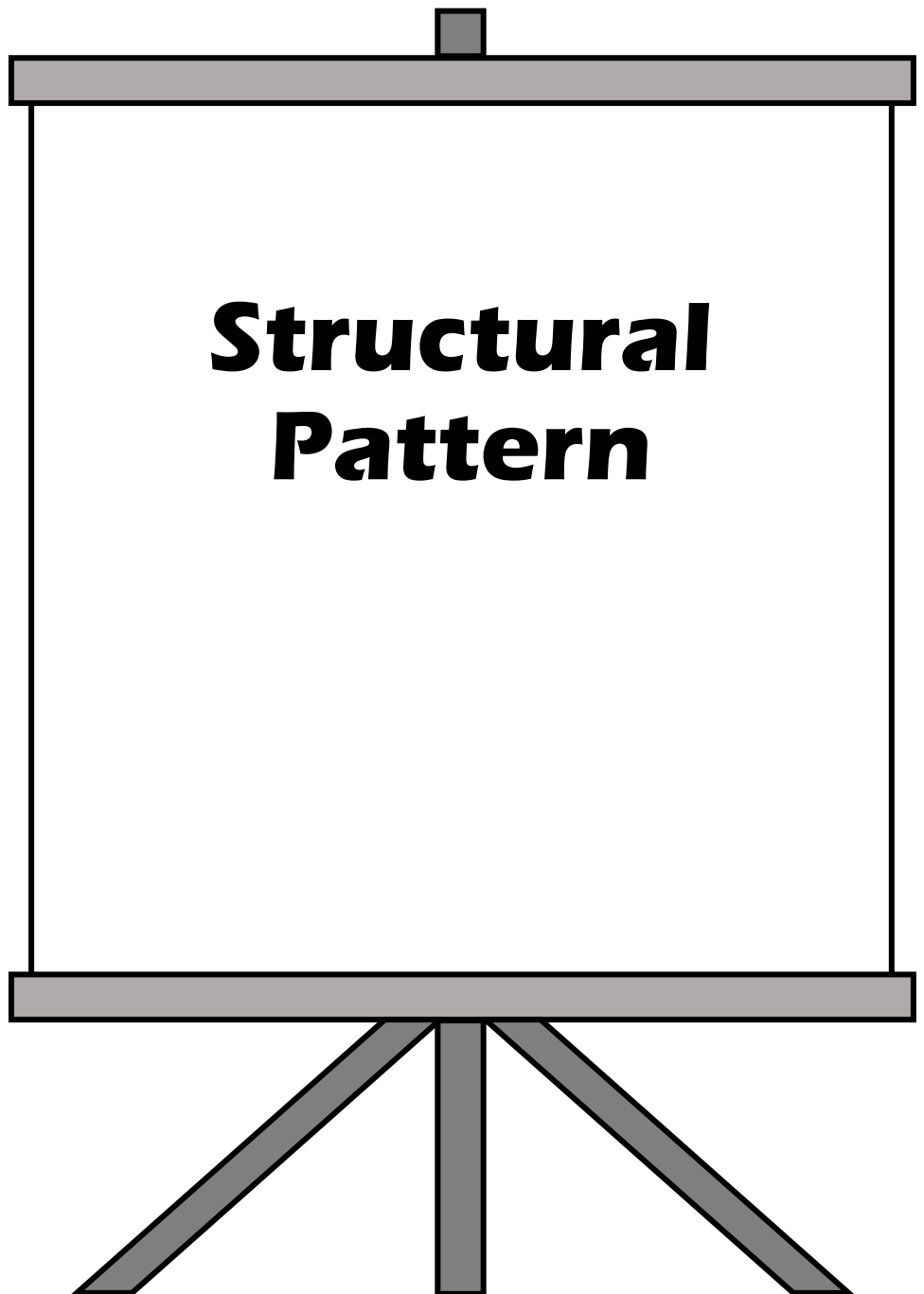
It provides a step-by-step approach to construct objects, allowing the construction process to vary independently from the object structure, resulting in a more flexible and scalable design.

Example: Building complex objects like a pizza with various toppings, where the builder pattern allows different combinations of toppings to be added in a stepwise manner.

7.2

LEARNING OUTCOMES

- ☑ Understanding of Structural Relationships: Learning structural patterns enhances understanding of relationships between classes or objects within a software system, such as composition, aggregation, and inheritance. This understanding helps in designing systems that are flexible, scalable, and maintainable.
- ☑ Ability to Design Flexible Architectures: Studying structural patterns equips individuals with the ability to design architectures that can accommodate changes in requirements and promote code reuse. By leveraging structural patterns effectively, developers can create modular, loosely coupled systems that are easier to understand, extend, and maintain over time.



Definition

focuses on the composition of classes or objects to form larger structures while keeping them flexible and efficient



Structural patterns are a category of design patterns that focus on the composition of classes or objects, aiming to form larger structures while keeping them flexible and efficient. These patterns help ensure that if one part of a system changes, the entire system doesn't need to do the same. They deal with relationships between classes or objects, providing solutions for organizing code, managing relationships, and improving system architectures.



Adapter Pattern

allows incompatible interfaces to work together by providing a wrapper or adapter class that converts the interface of a class into another interface that a client expects.



Bridge Pattern

separates an abstraction from its implementation so that both can vary independently. It involves creating two separate class hierarchies, one for the abstraction and one for the implementation, and then connecting them with a bridge interface



Composite Pattern

composes objects into tree structures to represent part-whole hierarchies. It allows clients to treat individual objects and compositions of objects uniformly.



DecoratorPattern

adds additional behavior or responsibilities to objects dynamically by wrapping them with decorator objects. It provides a flexible alternative to subclassing for extending functionality.



Facade Pattern

provides a unified interface to a set of interfaces in a subsystem. It simplifies the complex interactions between multiple classes or subsystems by providing a higher-level interface.



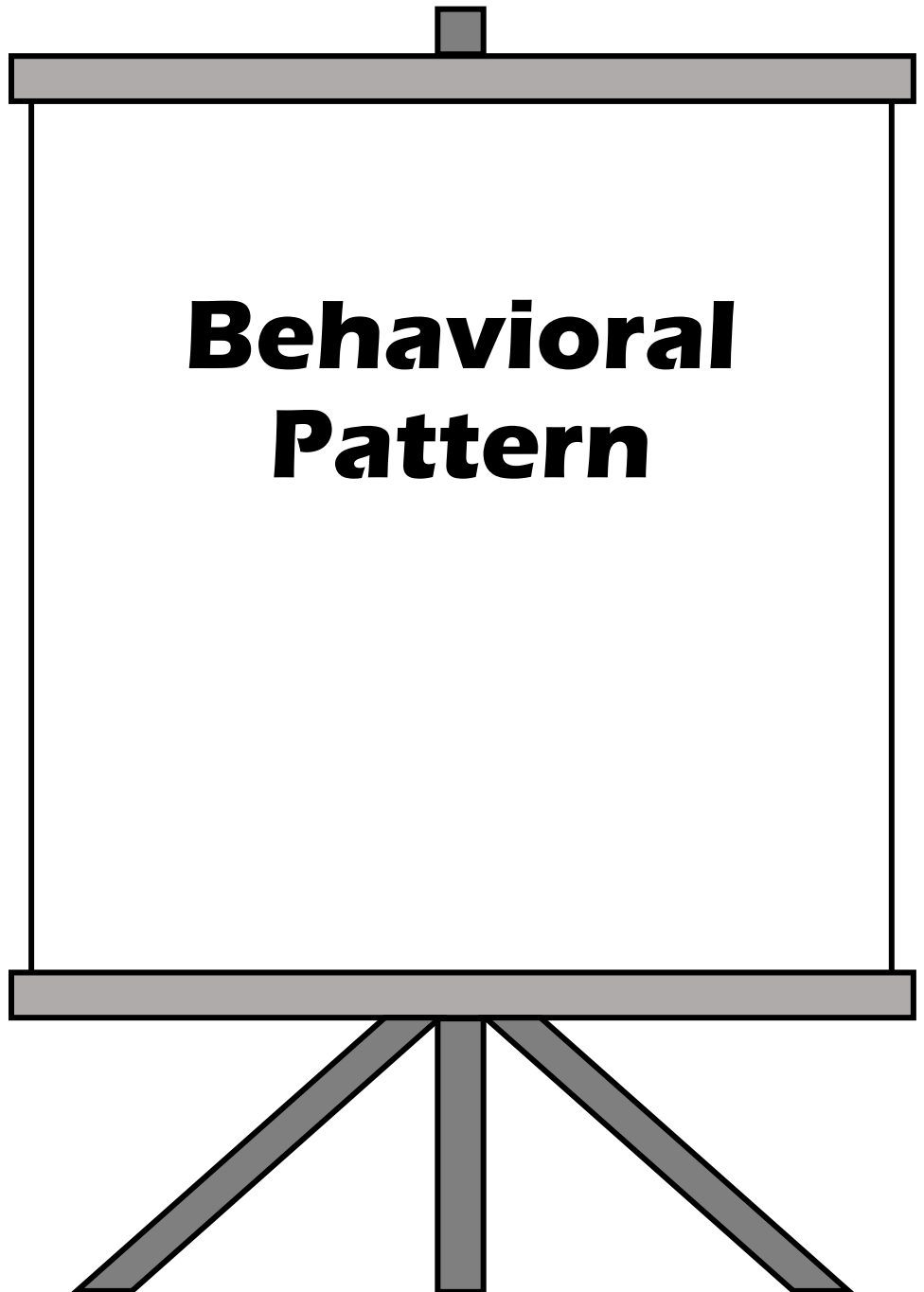
Composite Pattern

provides a surrogate or placeholder for another object to control access to it. It allows for additional functionalities, such as lazy initialization, access control, or logging, without changing the original object's interface

7.3

LEARNING OUTCOMES

- ❑ Understanding of Object Interaction: Behavioral patterns provide insights into how objects interact and collaborate within a system, enabling developers to design more flexible and maintainable software architectures. By studying behavioral patterns, individuals gain a deeper understanding of various communication mechanisms between objects, such as observer relationships, command invocations, or state transitions.
- ❑ Enhanced Problem-Solving Skills: Learning behavioral patterns enhances developers' problem-solving skills by providing them with a toolbox of proven solutions to common design challenges. By understanding when and how to apply behavioral patterns, individuals can effectively address issues related to object communication, state management, and algorithmic variability, leading to more robust and adaptable software designs.



Definition

software engineering describe communication and collaboration between objects in a system to accomplish various tasks, promoting flexibility and maintainability



Behavioral patterns are design patterns that focus on how objects collaborate and interact with each other to fulfill tasks and responsibilities. They capture the interactions between objects and the distribution of responsibilities among them. These patterns help in defining the communication between objects in a system and provide solutions for managing algorithms, responsibilities, and communication patterns.



Observer Pattern

Defines a one-to-many dependency between objects, so that when one object changes state, all its dependents are notified and updated automatically. Example: A weather station broadcasting weather updates to multiple display devices.



Strategy Pattern

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it. Example: Different sorting algorithms (bubble sort, quicksort) can be encapsulated as strategies for a sorting class.



Template Method Pattern

Defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps of the algorithm without changing its structure. Example: A framework for building different types of documents where the steps for generating the document are defined in a template method, with specific steps overridden by subclasses



Command Pattern

Encapsulates a request as an object, thereby allowing parameterization of clients with queues, requests, and operations. Example: A remote control that sends commands to various electronic devices such as TVs or stereos



State Pattern

Allows an object to alter its behavior when its internal state changes. The object will appear to change its class. Example: A vending machine that changes its state based on the number of coins inserted, allowing different actions for different states (e.g., sold out, has coins, etc.).



Iterator Pattern

Provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation. Example: Iterating through elements of a collection (like a list or array) without needing to know its internal structure.



Chain of Responsibility Pattern

Allows an object to send a command without knowing which object will receive and handle it. Example: A request in a web application passing through multiple middleware components, each capable of processing the request or passing it to the next component in the chain.



Interpreter Pattern

Defines a grammar for a language and provides an interpreter to interpret sentences in the language. Example: SQL queries being parsed and interpreted by a SQL interpreter to execute database operations



**NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES**

Score

Learning Activity / Exercise # ____
Design Pattern

Name

**Program-Year
Level**

Date

In the context of developing a software system for managing passenger traffic and operations at an MRT station, the student must be able to apply design patterns effectively.

Design patterns should be

1. utilized to address various challenges encountered in the development process, such as managing the flow of passengers through ticketing gates,
2. optimizing train scheduling and routing, and ensuring efficient communication between different subsystems of the station management software.

By employing design patterns such as

- Observer for monitoring passenger flow,
- Strategy for handling train scheduling algorithms, and
- Facade for simplifying interactions between subsystems,

the student can create a robust and maintainable software solution tailored to the specific requirements of an MRT station environment.

MODULE GOALS

- ☑ Understanding of GUI Development: Learning Java Swing provides a comprehensive understanding of graphical user interface (GUI) development in Java, including concepts such as components, layouts, event handling, and rendering.
- ☑ Proficiency in Creating Desktop Applications: Mastery of Java Swing enables developers to create rich and interactive desktop applications with a wide range of functionalities, empowering them to design user-friendly interfaces for various purposes, from simple utilities to complex enterprise applications.

College of Computing Information
Technologies

FLEX Course Material



Java Swing

**Definition**

a set of graphical user interface (GUI) components for Java applications, providing a comprehensive framework for building desktop applications.



Java Swing is a set of graphical user interface (GUI) components for Java applications, providing a comprehensive framework for building desktop applications. It includes a rich set of components such as buttons, text fields, labels, and tables, as well as layout managers for arranging these components on a window. Swing allows developers to create platform-independent GUIs that can run on any system with a Java Virtual Machine (JVM), making it a versatile and widely used toolkit for desktop application development in Java.



Creational Patterns

These patterns focus on the process of object creation, providing flexible ways to create objects while hiding the creation logic and making the system independent of how its objects are created, composed, and represented. Examples include Singleton, Factory Method, Abstract Factory, Builder, Prototype.



Structural Patterns

These patterns deal with the composition of classes or objects, focusing on the relationship between classes or objects to form larger structures. They help ensure that if one part of a system changes, the entire system doesn't need to do the same. Examples include Adapter, Bridge, Composite, Decorator, Facade, Proxy.

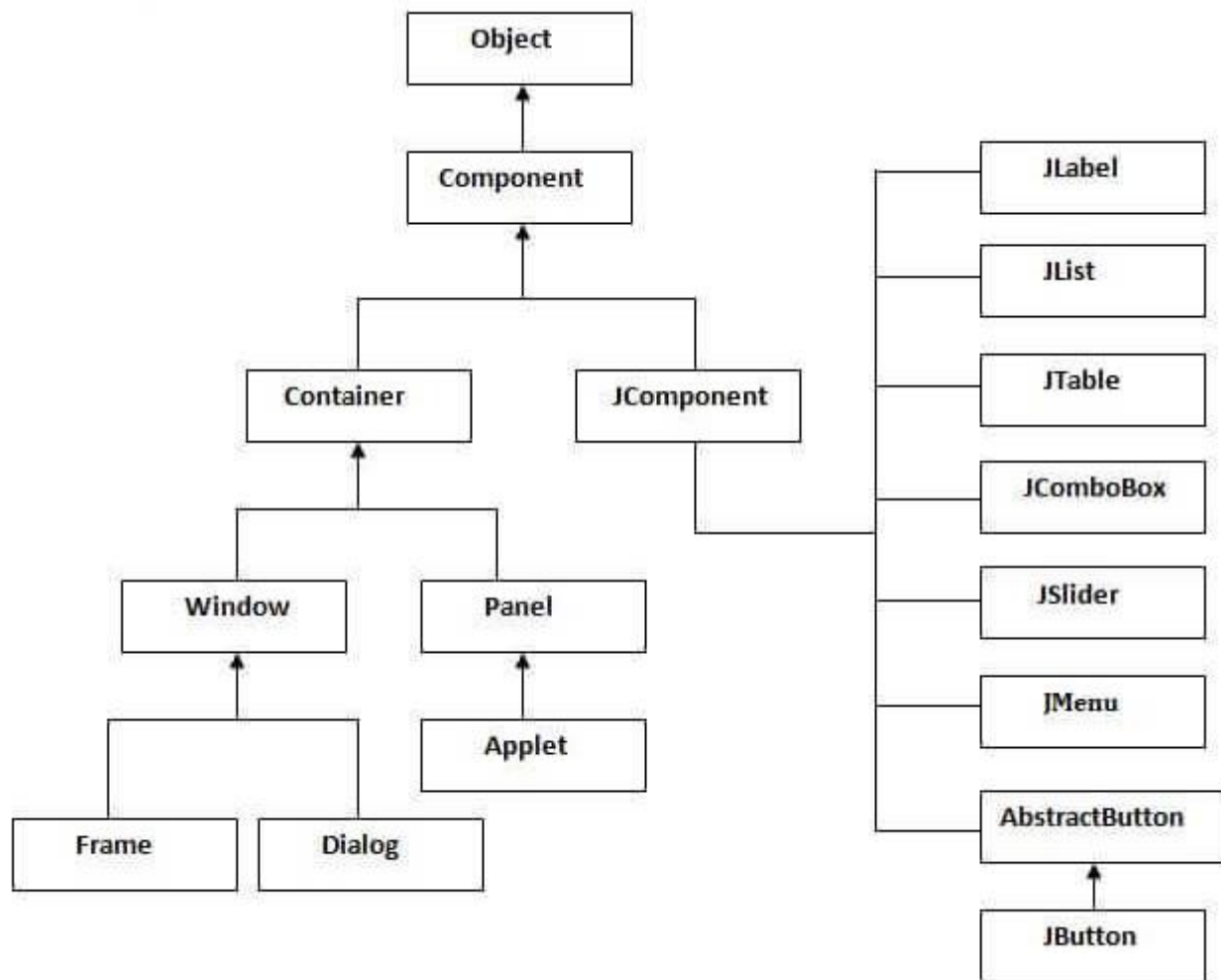


Behavioral Patterns

These patterns focus on the interaction between objects and the responsibility of objects, how they communicate, and how they collaborate to complete a task. They help in defining how objects interact in a loosely coupled way, promoting flexibility and maintainability. Examples include Observer, Strategy, Template Method, Command, State, Iterator.



The Java Swing class hierarchy provides a rich set of components for building graphical user interfaces (GUIs)



Source: <https://www.javatpoint.com/java-swing>



Root

java.awt.Component: This is the root class for all Swing components. It defines basic properties like size, position, visibility, and event handling.



Core Components

javax.swing.JComponent: This is a subclass of Component and the base class for most Swing UI components. It provides features like layout management, borders, and custom painting.



The Java Swing class hierarchy provides a rich set of components for building graphical user interfaces (GUIs) cont.



Core Components (cont.)

javax.swing.JLabel: A component to display text or an icon.
javax.swing.JButton: A button that can be clicked to trigger an action.
javax.swing.JTextField: A single-line text field for user input.
javax.swing.JTextArea: A multi-line text area for user input.
javax.swing.JScrollPane: A container that allows scrolling for larger content.
javax.swing.JPanel: A lightweight container to group other Swing components and manage their layout.
javax.swing.JFrame: A top-level window with a title bar, border, and ability to hold other components.
javax.swing.JDialog: A modal dialog window that requires user interaction before continuing with the main window.



Menus and Toolbars

javax.swing.JMenuBar: The top menu bar that holds menus.
javax.swing.JMenu: A menu containing menu items.
javax.swing.JMenuItem: An individual item within a menu.
javax.swing.JToolBar: A bar that can hold buttons and other components.



Layout Managers

javax.swing.LayoutManager: An interface that defines how components are arranged within a container. Swing provides several built-in layout managers like:

- **FlowLayout:** Arranges components from left to right, wrapping to the next line if necessary.
- **BorderLayout:** Divides the container into five regions (north, south, east, west, and center) for placing components.
- **GridLayout:** Arranges components in a grid-like fashion with equal sizes.



Menus and Toolbars

javax.swing.JTable: A table component for displaying and manipulating tabular data.
javax.swing.JList: A list component for displaying a list of items.
javax.swing.JComboBox: A drop-down combo box that allows users to select an item from a list.
javax.swing.JProgressBar: A component for displaying progress information as a bar.



**NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES**

Score

Learning Activity / Exercise # ____
Java Swing

Name

**Program-Year
Level**

Date

Given our expertise in developing payment observation systems for LRT/LRT2/MRT3 stations, let's proceed to design and implement a professional graphical user interface (GUI) tailored specifically for this purpose.

MODULE GOALS

- Understanding of Database Interaction: Learning Java database connection (JDBC) provides a solid understanding of how Java applications interact with relational databases, including concepts such as establishing connections, executing SQL queries, retrieving and manipulating data, and managing transactions.
- Proficiency in Database Integration: Mastery of JDBC enables developers to seamlessly integrate Java applications with various database systems, allowing them to build robust, data-driven applications that store, retrieve, and process information efficiently.



Java Database Connection

**Definition**

a Java API for connecting and interacting with relational databases.



Java Database Connectivity (JDBC) is a Java API for connecting and interacting with relational databases. It provides a standard interface for Java applications to access and manipulate data stored in databases using SQL (Structured Query Language). With JDBC, developers can establish connections to databases, execute SQL queries, retrieve and update data, and manage transactions, making it an essential tool for building database-driven applications in Java.

To connect a Java program to a MySQL database managed by XAMPP, you can use JDBC (Java Database Connectivity). Here's a step-by-step guide to achieve this:

1. Download and Install XAMPP:
 - First, download and install XAMPP from the official website: <https://www.apachefriends.org/index.html>
 - Follow the installation instructions provided for your operating system.
2. Start MySQL Server:
 - After installing XAMPP, start the MySQL server from the XAMPP Control Panel.
 - Make sure the MySQL service is running.
3. Set Up Database and User:
 - Use phpMyAdmin, which comes bundled with XAMPP, to create a database and a user with appropriate permissions.
 - Launch phpMyAdmin from the XAMPP Control Panel and create a new database (e.g., "mydatabase") and a user with privileges to access this database.
4. Download JDBC Driver:
 - Download the MySQL JDBC driver (mysql-connector-java.jar) from the MySQL website: <https://dev.mysql.com/download/s/connector/j/>
 - Add the downloaded JDBC driver JAR file to your Java project's classpath.



Sample Java to Xampp Connection

```
import java.sql.*;

public class DatabaseConnection {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/mydatabase"; // JDBC URL for the MySQL database

        String user = "username"; // Replace with your database username

        String password = "password"; // Replace with your database password

        try {

            // Establish connection

            Connection connection = DriverManager.getConnection(url, user, password);

            System.out.println("Connected to the database!");

            // Perform database operations (e.g., execute queries, update records, etc.)

            // Close connection
            connection.close();

            System.out.println("Connection closed.");

        } catch (SQLException e) {

            e.printStackTrace();

        }

    }

}
```



Java Database Connectivity (JDBC) is a Java API for connecting and interacting with relational databases. It provides a standard interface for Java applications to access and manipulate data stored in databases using SQL (Structured Query Language). With JDBC, developers can establish connections to databases, execute SQL queries, retrieve and update data, and manage transactions, making it an essential tool for building database-driven applications in Java.

To connect a Java program to Microsoft SQL Server (MSSQL) using JDBC, you need to follow these steps::

1. Download JDBC Driver for MSSQL:
 - First, download the JDBC driver for MSSQL from the Microsoft website or Maven repository. The JDBC driver is a JAR file that contains the necessary classes for connecting to MSSQL databases.
2. Include JDBC Driver in Your Java Project:
 - After downloading the JDBC driver JAR file, include it in your Java project's build path or dependencies.

```
import java.sql.*;

public class MSSQLConnectionExample {
    public static void main(String[] args) {
        // Database URL
        String url =
"jdbc:sqlserver://localhost:1433;databaseName=mydatabase";

        // Database credentials
        String username = "your_username";
        String password = "your_password";

        // Connection object
        Connection conn = null;

        try {
            // Register JDBC driver
            Class.forName("com.microsoft.sqlserver.jdbc.S
QLServerDriver");
```



Sample Java to Xampp Connection

```
// Open a connection
        conn = DriverManager.getConnection(url,
username, password);

        // Execute a query
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM
my_table");

        // Process the result set
        while (rs.next()) {
            // Retrieve data from the result set
            int id = rs.getInt("id");
            String name = rs.getString("name");
            // Display data
            System.out.println("ID: " + id + ", Name: " +
name);
        }

        // Close the result set, statement, and connection
        rs.close();
        stmt.close();
        conn.close();
    } catch (SQLException | ClassNotFoundException e) {
        // Handle exceptions
        e.printStackTrace();
    } finally {
        // Close the connection in the finally block
        try {
            if (conn != null) { conn.close(); }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```



NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES



Score

Learning Activity / Exercise # ____
Java Database Connection

Name

Program-Year
Level

Date

Given our expertise in developing payment observation systems for LRT/LRT2/MRT3 stations, let's proceed to design and implement a professional graphical user interface (GUI) with database connection tailored specifically for this purpose.



Expected Output

Title: The Great Gatsby

Author: F. Scott Fitzgerald

Pages: 180



NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES



Score

Learning Activity / Exercise # ____
Encapsulation – Sample Source Code

Name

Program-Year
Level

Date

```
// Book.java
class Book {
    private String title;
    private String author;
    private int pages;

    // Constructor to initialize title, author, and number of pages
    public Book(String title, String author, int pages) {
        this.title = title;
        this.author = author;
        this.pages = pages;
    }

    // Getter method to retrieve the title of the book
    public String getTitle() {
        return title;
    }

    // Getter method to retrieve the author of the book
    public String getAuthor() {
        return author;
    }

    // Getter method to retrieve the number of pages of the book
    public int getPages() {
        return pages;
    }
}

// Main.java
public class Main {
    public static void main(String[] args) {
        // Create a new book object
        Book book = new Book("The Great Gatsby", "F. Scott Fitzgerald", 180);

        // Accessing properties using getter methods
        System.out.println("Title: " + book.getTitle());
        System.out.println("Author: " + book.getAuthor());
        System.out.println("Pages: " + book.getPages());
    }
}
```



Score

Learning Activity / Exercise # ____
Inheritance – Sample Source Code

Name

Program-Year
Level

Date

```
// Shape class (Parent Class)
class Shape {
    // Method to calculate area (to be overridden by
    subclasses)
    public double calculateArea() {
        return 0.0;
    }

    // Method to calculate perimeter (to be overridden by
    subclasses)
    public double calculatePerimeter() {
        return 0.0;
    }
}

// Rectangle class (Subclass of Shape)
class Rectangle extends Shape {
    private double length;
    private double width;

    // Constructor
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    // Override calculateArea method
    @Override
    public double calculateArea() {
        return length * width;
    }

    // Override calculatePerimeter method
    @Override
    public double calculatePerimeter() {
        return 2 * (length + width);
    }
}

// Circle class (Subclass of Shape)
class Circle extends Shape {
    private double radius;
```

```
// Constructor
public Circle(double radius) {
    this.radius = radius;
}

// Override calculateArea method
@Override
public double calculateArea() {
    return Math.PI * radius * radius;
}

// Override calculatePerimeter method
@Override
public double calculatePerimeter() {
    return 2 * Math.PI * radius;
}

// Main class to test the shape hierarchy
public class Main {
    public static void main(String[] args) {
        // Create a rectangle
        Rectangle rectangle = new Rectangle(5, 3);
        System.out.println("Rectangle Area: " +
        rectangle.calculateArea());
        System.out.println("Rectangle Perimeter: " +
        rectangle.calculatePerimeter());

        // Create a circle
        Circle circle = new Circle(4);
        System.out.println("Circle Area: " +
        circle.calculateArea());
        System.out.println("Circle Perimeter: " +
        circle.calculatePerimeter());
    }
}
```



Score

Learning Activity / Exercise # ____ Abstraction

Name	Program-Year Level	Date
<pre>// Shape interface interface Shape { void drawWith(Tool tool); } // Tool interface interface Tool { void drawCircle(Circle circle); void drawSquare(Square square); } // Concrete implementation of Tool interface: Pen class Pen implements Tool { @Override public void drawCircle(Circle circle) { System.out.println("Drawing circle with pen"); } @Override public void drawSquare(Square square) { System.out.println("Drawing square with pen"); } } // Concrete implementation of Tool interface: Brush class Brush implements Tool { @Override public void drawCircle(Circle circle) { System.out.println("Drawing circle with brush"); } @Override public void drawSquare(Square square) { System.out.println("Drawing square with brush"); } }</pre>	<pre>// Circle class implementing Shape interface class Circle implements Shape { @Override public void drawWith(Tool tool) { tool.drawCircle(this); // Double dispatch occurs here } } // Square class implementing Shape interface class Square implements Shape { @Override public void drawWith(Tool tool) { tool.drawSquare(this); // Double dispatch occurs here } } // Main class public class Main { public static void main(String[] args) { Shape circle = new Circle(); Shape square = new Square(); Tool pen = new Pen(); Tool brush = new Brush(); circle.drawWith(pen); // Output: Drawing circle with pen circle.drawWith(brush); // Output: Drawing circle with brush square.drawWith(pen); // Output: Drawing square with pen square.drawWith(brush); // Output: Drawing square with brush } }</pre>	